

Universitatea
Transilvania
din Brașov

FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

PROIECT DE DIPLOMĂ

Conducători științifici:

conf. dr. ing. Danciu Gabriel Mihail

șef lucr. dr. ing. BOGDAN Ioana Corina

Absolvent:

Gîrbacea Marian Ionuț

Brașov, 2026

Departamentul de Electronică și Calculatoare
Programul de studii: Calculatoare

GÎRBACEA Marian Ionuț

Sistem interactiv de antrenare VR în mediul industrial cu roboți

Conducători științifici:

conf. dr. ing. DANCIU Gabriel Mihail

șef lucr. dr. ing. BOGDAN Ioana Corina

Brașov, 2026

Cuprins

Lista de figuri, tabele și coduri sursă	5
Lista de acronime	6
1 Context și motivație	6
1.1 Context	6
1.2 Motivație	6
2 Obiectivele lucrării	8
3 Fundamente teoretice	9
3.1 Digital Twin	9
3.2 Cinematica inversă	9
3.3 Realitatea virtuală	9
3.4 MQTT	10
3.5 URDF	11
3.6 Roboții industriali	11
3.7 Tunneling	12
4 Tehnologii și aplicații folosite	13
4.1 Unity	13
4.1.1 "Unity XR Interaction Toolkit"	13
4.1.2 "Articulation Body"	14
4.1.3 "RayCast"	14
4.1.4 "RigidBody"	14
4.1.5 "Unity URDF Importer"	14
4.2 Biblioteci și clase folosite	15
4.2.1 Biblioteci folosite în Unity	15
4.3 Clasa "Monobehaviour"	15
4.4 Biblioteci braț robotic real	16
4.5 Fusion 360	16
4.6 Ngrok	16
5 Echipamente folosite	17
5.1 MaxArm	17
5.2 Oculus Quest 3S	19
6 Arhitectura sistemului	20
6.1 Nivelul de simulare și interacțiune	20
6.2 Nivelul de comunicație	20
6.3 Nivelul de procesare al robotului fizic	21
6.4 Nivelul de execuție hardware	21
7 Implementări	23
7.1 Realizarea modelului 3D al brațului robotic	23
7.2 Mediul din Unity	28
7.3 Importarea în Unity	28
7.4 Setarea proprietăților	29

7.4.1	Compensări cinematice	30
7.5	Sistemul de control al brațului în Unity	31
7.5.1	Metoda de control bazată pe control direct	32
7.5.2	Metoda de control bazată pe cinematică inversă	32
7.6	Resetarea robotului	34
7.7	Manevrarea obiectelor	34
7.8	Sistemul de antrenare pentru operatorul uman	35
7.8.1	Lista cu controale	35
7.8.2	Lista cu instrucțiuni de bază	36
7.9	Vizualizarea robotului din realitate în mediul virtual	37
7.10	Procesul de comunicație de date dintre Unity și cel din realitate	37
7.10.1	Comunicația din Unity	38
7.10.2	Comunicația din robotul real	40
8	Testare și rezultate	42
8.1	Importanța rezultatelor experimentale	42
8.2	Testarea funcționalității-lor	42
8.2.1	Testarea constrângerilor cinematice și a bazei secundare rotative	42
8.2.2	Testarea controlului brațului virtual folosind primul mod de control	42
8.2.3	Testarea controlului brațului folosind cel de al doilea mod de control	43
8.2.4	Testarea funcționalității de reset și manipulare de obiecte	43
8.2.5	Testarea sistemului de tutorial	43
8.2.6	Testarea fluxului de comunicație între brațe	44
8.3	Interpretarea rezultatelor testelor	44
8.3.1	Metrici, metri/secunda, filmand pas cu pas si script separat unde dau pasi de miscare gen coborare/urcare. fata/spate	45
	Bibliografie	46

FIGURI

1	(Brațul Robotic MaxArm)	18
2	(Quest3S)	19
3	(Arhitectură sistem)	22
4	(Model 3D baza brațului)	23
5	(Model 3D bază secundară)	23
6	(Model 3D segment vertical principal)	24
7	(Model 3D segment orizontal principal)	24
8	(Model 3D segment lateral de urcare-coborâre)	24
9	(Model 3D tijă susținere 1)	25
10	(Model 3D tijă ajutătoare verticală 2)	25
11	(Model 3D tijă ajutătoare orizontală 1)	25
12	(Model 3D tijă ajutătoare orizontală 2)	26
13	(Model 3D suportul pompei de sucțiune)	26
14	(Model 3D ansamblul pentru ventuza de sucțiune)	26
15	(Model 3D)	27
16	(Structura ierarhica Unity)	29

TABELE

CODURI SURSĂ

2D - 2 Dimensiuni;
3D - 3 Dimensiuni;
A - Amperi;
AR - Augumented Reality;
CAD - Computer Aided Design;
CAE - Computer Aided Engineering;
cm - centimetrii;
DC- Direct Current;
DDos - Distributed Denial of Service;
Kg - Kilograms;
CAM - Computer Aided Manufacturing;
mA - miliAmperi
mm-milimetrii;
PPD - Pixel Per Degree;
PPI - Pixel Per Inch;
PWN - Pulse Width Modulation;
RGB - Red Green Blue;
HMD - Head-Mounted Display;
HZ - Hertz;
ID - Identification;
IoT - Internet of Things;
MQTT - Message Queuing Telemetry Transport;
OASIS - Organization of Advancement of Structured Information Standard;
SCARA - Selective Compliance Articulated Robot Arm;
STL -Stereolithography;
TCP - Transmission Control Protocol;
TMP - TextMeshPro;
UI - User Interface;
URDF - Unified Robotics Description Format;
UDP - User Datagram Protocol;
USB - Universal Serial Bus;
XML - Extensible Markup Language;
XR - Extended Reality;
V - Voltage;
VR - Virtual Reality;
WIFI - Wireless Fidelity;

1.1 Context

În societatea contemporană majoritatea domeniilor cum sunt cele de producție, farmaceutică până la apărare sau alimentar prezența echipamentelor de tip roboți sau brațe robotice a devenit tot mai răspândită, unde se integrează expertiza umane cu cea a asistenței din partea roboților. Astfel a apărut o nevoie pentru testarea și găsirea mai rapidă a erorilor și defectelor unor astfel de echipamente din partea producătorilor și antrenarea directă pe aceștia poate duce la defectarea lor sau chiar la accidentări, ducând la costuri suplimentare atât din punct de vedere al companiei care a achiziționat robotul cât și din partea producătorului robotului.

O soluție pentru testarea și găsirea eventualelor erori și defecte o reprezintă tehnologia "digital twin". Pentru partea de antrenare a operatorului uman se poate folosi realitatea virtuală pentru a implementa un mediu de antrenament ce implementează un "digital twin", mediul cuprinzând majoritatea scenariilor în care operatorul uman și brațul robotic pot să fie supuse în activitățile la care sunt implicate.

O interacțiune directă dintre robotul real și "digital twin" poate fi utilă în momentul în care este necesară controlul direct al brațului real sau al robotului în medii periculoase cum sunt cele manevrarea elementelor radioactive sau a fiolelor din laboratoare.

Lucrarea își propune realizarea unui astfel de mediu de antrenament în realitatea virtuală folosind implementarea unui "digital twin" pentru un braț robotic cu 4 grade de libertate ce permite ridicarea și mutarea de obiecte folosind o pompă. De asemenea acest sistem urmărește ca procesul de antrenare a utilizatorilor umani, care nu au interacționat niciodată sau rar cu un astfel de sistem să fie cât mai simplă, intuitivă astfel încât timpul petrecut în antrenare să fie cât mai mic și nivelul de pregătire dobândit să fie cât mai mare.

1.2 Motivație

Motivația principală din spatele realizării constă în creșterea în popularitate a tehnologiei de "digital twin" în domenii precum: medical, de producție domeniul energetic [1]. Pe partea de producție aceștia pot ajuta operatorul să înțeleagă condițiile asset-ului și să realizeze planuri predictive pentru mentenanță. Producătorii folosind "digital twins" pentru planificarea zonei de producție, pot să simuleze impactul asupra liniei de producție, ajustând și optimizând pentru cel mai bun flux de lucru. În domeniul auto, producătorii deja folosesc "digital twins" pentru a prototipa și simula modele noi de vehicule. Aceștia pot să ofere posibilitatea pentru vehicule pe bază de software și mașini autonome [2]. Pentru domeniul energetic, "digital twins" pot fi folosiți în rețele electrice și tehnologia microgrid, pentru a ajuta operatorii în optimizarea și distribuția de energie. Flexibilitatea acestei tehnologii le oferă operatorilor noi unelte pentru a răspunde într-un mod proactiv la deranjamente, să simuleze situații de urgență și în planuirea integrării surselor de energie regenerabilă eficientă din punct de vedere al costului. În medicină, le oferă furnizorilor de servicii medicale posibilitatea îmbunătățirii uneltelor de monitorizarea a pacienților. Prin combinarea datelor de la dispozitive multiple destinate pacienților într-un "digital twin", furnizorii pot să obțină un o vedere mai mare de ansamblu asupra sănătății pacientului. De asemenea, pot ajuta administratorii să monitorizeze rata de ocupare și îngrijorările legate de siguranță sau chiar să monitorizeze calitatea aerului pentru a ajuta în prevenirea răspândirii patogenilor [2].

Un alt domeniu în care această tehnologie poate fi folosită este cea a telecomunicațiilor. Aceasta le poate oferi furnizorilor din telecom să obțină o imagine de ansamblu mai bună asupra infrastructurii rețelei, a stării asset-ului și să ofere posibilitatea de a se realiza mentenanță predictivă pentru minimizarea momentelor de picare a rețelei. Pe lângă furnizorii îmi mai pot folosi în simularea fluctuării

cererilor din rețea pentru teste de stres sau planificarea alternativelor fără necesitate de schimbări costisitoare în lumea reală [2].

Un avantaj principal al acestui proiect constă în folosirea tehnologie de realitate virtuală, care reprezintă o soluție modernă care permite simularea și controlul acestor roboți, oferindu-le utilizatorilor posibilitatea de a învăța cum să-i controleze, limitele lor și scenariile de utilizare fără a fi necesar interacțiunea fizică cu cei reali. Un sistem în realitatea virtuală poate reduce costurile, crește eficiența în antrenare și de a îmbunătăți nivelul de pregătire fără a afecta productivitatea reală.

Un alt aspect important al proiectului constă în implementarea a 2 metode de control al brațului din VR. Această metodă de implementare a metodei de control prezintă un avantaj semnificativ deoarece oferă utilizatorului posibilitatea de control al robotului într-un mod simplu și ușor de înțeles, cât și producătorului să testeze rapid implementări și să rezolve eventualele probleme apărute.

Principalele obiective stabilite pentru realizarea lucrării:

- Dezvoltarea unui sistem de antrenament care să ofere un minim de pregătire asupra controlului robotului
- Realizarea modelului 3D al brațului din realitate
- Importarea modelului 3D al brațului în mediul VR din Unity
- Aplicarea proprietăților și constrângerilor robotului care sunt necesare pentru mișcarea acestuia
- Realizarea primului mod de mișcare al robotului baza pe asociere directă între butoanele de la manete și motoare
- Realizarea celui de al 2 lea mod de control al robotului bazat pe cinematică inversă
- Aplicarea valorilor citite din Unity asupra robotului din lumea reală pentru a putea fi controlat
- Primirea feedback-ului de la robotul real legat de valorile pe care le-a recepționat

3.1 Digital Twin

Un "digital twin" este o reprezentare virtuală a unui obiect fizic sau sistem ce folosește date din timp real să reflecte cu acuratețe comportamentului geamănului din lumea reală, performanțe și condiții. "Digital twins" permit monitorizate continuă, simulare și analiză a unui obiect, sistem sau produs pe durata timpului său de viață, de la proiectare și producție, până la mentenanță și dezafectare. Aceștia pot să reprezinte orice obiect într-un mod virtual, de la clădiri și poduri, până la mașini, avioane, artefacte antice și chiar întreaga planetă[3].

De asemenea, prin "digital twins" pot să se modeleze sisteme complexe cum sunt: traficul, evenimente meteorologice, planuri medicale de tratament, procese și operațiuni din fabrici sau în medii experimentale, și chiar pot să fie bazați fie pe oameni reali, fie imaginari cu toate trăsăturile necesare cum sunt: tonul vocii, înfățișare și personalitate [3].

3.2 Cinematica inversă

Cinematica reprezintă studiul mișcării fără să se considere cauza mișcării, cum sunt forțele și cuplurile. Cinematica inversă reprezintă folosirea ecuațiilor cinematice să determine mișcarea unui robot pentru a ajunge la destinație. De exemplu, pentru a putea apuca un obiect, un braț robotic folosit în procese de fabricație necesită mișcări precise de la poziția inițială către poziția necesară. Capătul de prindere al robotului reprezintă efectorul final. Configurația robotului reprezintă o listă a pozițiilor de "Joint-uri" care pot fi acționate atât timp cât nu depășesc constrângerile robotului.

Pornind de la poziția la care efectorul final trebuie să ajungă, cinematica inversă poate să determine o configurație a "Joint-urilor" astfel încât efectorul final să poată să se miște către țintă [4].

3.3 Realitatea virtuală

Realitatea virtuală reprezintă un termen care descrie un mediu 3D generat de un calculator care poate fi explorat și se poate interacționa de către un utilizator uman. Acel utilizator devine o parte a mediului virtual sau devine imersiv cu mediul și cât timp se află în mediu, poate să manipuleze obiectele sau să performeze o serie de acțiuni.

Realitatea virtuală este implementată folosind sisteme care sunt destinate pentru realizarea mediului, cum sunt căștile, benzile de mers multi-direcționale și mănuși speciale. Aceste sisteme sunt folosite să stimuleze simțurile simultan pentru a crea o iluzie a realității

Această tehnologie poate fi împărțită în 5 tehnologii principale:

- Realitatea virtuală non-imersivă se referă la utilizarea unor serii de ecrane care înconjoară utilizatorul pentru ai fi prezentată informația virtuală[5].
- Realitatea virtuală semi-imersivă în care componentele digitale se suprapun peste obiectele reale. Astfel aceste elemente virtuale pot fi folosite într-un mod similar cu cele reale [6].
- Realitatea virtuală imersivă, care a fost folosită în acest proiect se folosește de folosirea unui HMD pentru a urmări mișcările utilizatorului și să prezinte informația VR în funcție de mișcările utilizatorului[5].
- Realitatea augmentată care reprezintă un alt mod de a realiza realitate virtuală prin modul în care dezvoltatorii suprapun cele ambele realități [6].

- Realitate mixtă combină lumea fizică cu cea virtuală. Reprezintă un caz special de realitate augmentată. Această tehnologie permite vizualizarea de oameni și obiecte într-un context real [6].

3.4 MQTT

MQTT reprezintă un standard OASIS fiind un protocol de mesagerie folosit în IoT. Este proiectat ca să implementeze protocolul de tip publicare/abonare într-un mod în care să folosească cât mai puține resurse. Acest protocol este ideal pentru conectarea între dispozitive, ce sunt limitate din punct de vedere hardware. Comunicarea prin acest protocol este bidirecțională. [7].

Mesageria de tip publicare/abonare reprezintă un model de comunicare asincron care permite dezvoltatorilor să construiască aplicații funcționale și complexe în cloud. Pentru aceste sisteme cloud modelul de mesagerie oferă notificări instantanee când se întâmplă un eveniment în sistemele distribuite [8].

Acest model funcționează în felul următor:

- Mesajele reprezintă un set de date de comunicație care sunt trimise de la emițător la receptor. Mesajele pot fi de orice tip de dată [8]
- Topicul care acționează ca un canal intermediar între emițător și receptor. Gestionează o listă de receptori care sunt interesate în mesajele care au același topic [8]
- Abonații reprezintă destinatarul mesajului. Aceștia trebuie să se înregistreze sau să se aboneze la un topicul care îi interesează. Pot să performeze diferite funcții sau să facă ceva diferit cu mesajul în paralel [8]
- Publicatorii reprezintă componenta care trimite mesajele. Creează mesaje despre un topic și le trimite pe toate la toți abonații ai acelui topic. Interacțiunea dintre publicator și abonați reprezintă o relație de tipul unul la mulți. Publicatorul nu trebuie să știe cine folosește informația, iar abonații nu trebuie să cunoască proveniența mesajelor, transmisia fiind una de difuzare [8]

Modelul de mesagerie de tip publicare/abonare reprezintă următoarele avantaje:

- Eliminarea polling-ului. Topicurile de mesaje care permit transmiterea mesajelor instantaneu eliminând verificările periodice pe care consumatorii mesajului trebuie să le facă când apare informație nouă sau când este actualizată [8].
- Direcționare dinamică. Modelul publicare/abonat face ca descoperirea serviciilor mai ușor, mai natural, și mai greu susceptibilă la erori. În loc ca aplicația să întrețină un grup de abonați căruia să-i trimită mesajele, un publicator va posta mesajele către un topic specific, astfel încât abonații să se aboneze la mesajele cu topic-urile de interes [8].
- Simplificarea comunicației. Acest model simplifică procesul de comunicație prin eliminarea conexiunilor punct-la-punct care leagă o singură conexiune către un topic. Topicul va gestiona abonările, astfel încât să decidă ce mesaje sunt destinate cărui endpoint [8].

Aceste mesaje pot fi consumate de oricare număr de noduri, incluzând, filtre, sisteme de nivel înalt cum ar fi cele de ghidare [9].

3.5 URDF

URDF este un limbaj markup bazat pe XML, dezvoltat ca parte a ecosistemului ROS. Este folosit în a reprezenta modelul unui robot în raport cu: structura fizică, cinematica, proprietățile legate de inerție, aspectul fizual și geometria coliziunilor [10].

Un fișier definește un robot ca o colecție de legături (corpuri rigide), conectate prin articulații (care definește cum legăturile se pot mișca între ele). De asemenea poate specifica senzori, transmisii și materiale, permițând simularea și vizualizarea robotului în mediul 3D [10].

URDF servește ca o componentă esențială în controlul prin:

- Simulare: Modelele URDF sunt folosite pentru a încărca roboții în simulatoare cum este Gazebo, sau medii de dezvoltare 3D cum este Unity, unde dezvoltatorii testează algoritmi în medii de dezvoltare virtuale controlate înainte să fie încărcate pe hardware. URDF se asigură că constrângerile fizice și reprezentările vizuale se aseamănă cu cele din lumea reală [10].
- Cinematica și planificarea mișcărilor. Este necesară pentru a ajuta la înțelegerea limitelor articulațiilor și ierarhiei dintre legături și coordonatele componentelor. Acestea permit planificarea mișcărilor cu acuratețe și calculului cinematicii inverse și directe [10].
- Vizualizare și Debugging. URDF este folosit pentru a randa vizualizări 3D ale robotului în timp real. Se pot suprapune datele de la senzori pe model, ceea ce face mai ușor înțelegerea comportamentului sistemului și debugging-ul [10].
- Integrarea cu senzori. URDF permite poziționarea cu precizie a cadrelor și a pozițiilor senzorilor pe robot. Modelarea acestora cu acuratețe este esențială pentru combinarea senzorilor și a navigației [10].

3.6 Roboții industriali

Roboții industriali sunt folosiți în procese de asamblare, curățare și celelalte activități din fabrici unde oamenii nu pot opera. Automatizarea fiecărui proces îmbunătățește eficiența și contribuie la reducerea lipsei forței de muncă și al costurilor [11].

Roboții industriali se împart în 6 tipuri:

- Roboți liniari. Aceștia sunt asemănători în aspect cu o macara de tip pod cu 3 axe de mișcare, are o capacitate de mutare de obiecte și acuratețe ridicată. Axele acestui robot sunt perpendiculare între ele și robotul se mișcă cu axele în linie dreaptă [12].
- Roboți cilindrici. Aceștia sunt roboți pe 3 axe cu zone de lucru sub formă de cilindri, 2 axe ale robotului se mișcă linear, iar cel de al 3 lea se mișcă circular [12].
- Roboți sferici. Aceștia au sistem de coordonate polare. Au 2 axe de rotație și o axă lineară.
- Roboți SCARA. Sunt roboți de mare precizie și viteză ce au 4 axe de mișcare. Încheieturile roboților se mișcă pe un plan orizontal împreună cu alte 3 axe, și o altă componentă se mișcă sus și jos împreună cu cea de a 4 a axă [12].
- Roboți articulați. Sunt roboți pe 6 axe compuse din încheieturi conectate secvențial, care copiază aspectul unei mâini umane. Cele 6 axe oferă robotului flexibilitate și le permite axelor să ajungă în zone în care ceilalți nu pot [12].
- Robot delta. Sunt roboți ce au 3 mânere atașate de o bază și care țin componenta de lucru a robotului paralelă cu baza. Aceștia pot avea 3, 4 și 6 axe de mișcare. Varianta cu 3 axe poate muta obiectele pe planuri paralele, și variantele cu un număr mai mare de axe sunt capabil să rotească obiecte și să manipuleze obiecte din părți diferite [12].

3.7 Tunneling

Tunneling reprezintă o metodă pentru a transporta date între rețele folosind protocoale care nu sunt suportate de acele rețele. Funcționează prin încapsularea pachetelor. Tunneling este de obicei folosit în rețele virtuale private, și de asemenea poate să stabilească conexiuni eficiente și sigure între rețele [13].

4 Tehnologii și aplicații folosite

În acest capitol vor fi prezentate aplicațiile și tehnologiile folosite în implementarea proiectului.

4.1 Unity

Unity reprezintă o platformă interactivă de creare de conținut 3D, 2D pentru VR și AR. [14].

Acesta este un motor de creare de conținut în timp real pentru crearea de experiențe care sunt interactive într-un mod complet care este destinat atât utilizatorului final cât și pentru creatorii de conținut [14].

Termenul de timp real descrie rapiditatea cu care o imagine este randată sau afișată pe un ecran chiar și când aceasta este schimbată de către utilizator. Scopul programelor software de timp real este de a randa imaginile atât de rapid încât interacțiunea cu mediul virtual să se facă fără observarea acută a întârzierilor [14].

Unity are aplicații în următoarele domenii: media, divertisment, arhitectură inginerie, construcții, automotiv, transport și producție [14].

Unity în acest proiect a fost folosit pentru simplitatea cu care acest motor de creare a conținutului 3D integrează tehnologii de modelare de conținut 3D, cât și tehnologii de vizualizare și interacțiune cu roboții [14].

4.1.1 "Unity XR Interaction Toolkit"

"XR Interaction Toolkit" este un sistem de nivel înalt și bazat pe componente de interacțiune pentru crearea de aplicații VR și AR. Oferă un framework care poate fi folosit în dezvoltarea de interacțiuni de 3D și UI, acestea fiind disponibile din Unity Input Events. Centrul acestui sistem este bazat pe un interactor și componente interacționabile, și un gestionator de interacțiuni care leagă împreună aceste tipuri de componente. De asemenea conține componente care pot fi folosite pentru locomoție și desenare [15].

"XR Interaction Toolkit" conține următorul set de componente care suportă următoarele acțiuni de interacțiune:

- Compatibilitate cu o gamă înalte de platforme folosite pentru control: "Meta Quest, OpenXR, Windows Mixed Reality și altele" [15]
- Manipulare de bază a obiectelor din scenă [15]
- Feedback haptic prin manetele de XR [15]
- Feedback vizual pentru indicarea interacțiunilor active și posibile [15]
- Interacțiune utilă cu "XR Origin", ce reprezintă o componentă de cameră VR folosită în manevrarea staționară și în funcție de mărimea zonei a experiențelor VR [15]

4.1.2 "Articulation Body"

"Articulation body" reprezintă o componentă ce permite construirea de articulații fizice cum sunt brațele robotice, lanțuri cinematice cu "GameObjects" care sunt organizate într-un mod ierarhic de tipul părinte-copil sau arbore. Acestea ajută în obținerea de fizică comportamentală realistă în contextul simulărilor și aplicațiilor industriale [16].

Un "articulation body" permite definirea pentru un singur obiect proprietăți, care pot fi definite de alte componente asemănătoare cum sunt "RigidBody" sau "Regular Joint" într-o configurație clasică. Aceste proprietăți depind de tipul de poziția obiectului în ierarhie [16].

Acestea sunt:

- pentru obiectul rădăcină sau baza: masa, mutabilitatea, și dacă să folosească gravitația [16]
- pentru obiectele copil sau brațe: masa, dacă să folosească gravitația, tipul pe rotație al articulației, legarea de ancora părintelui, poziția și rotația ancorei, fricțiunea articulației, amortizarea liniară și unghiulară, gradele de libertate ale mișcării, limitele unghiulare superioare și inferioare, rigiditatea, amortizarea, limita forței, poziția unghiului și viteza de deplasare [16].

Articulațiile din "articulation body" sunt rezolvate folosind metoda "Featherstone" care lucrează folosind coordonate reduse prin: fiecare corp, copil sau componentă are coordonare relative față de părintele acestuia dar numai în jurul gradelor de libertate pe care le are setate. Acestea garantează că nu vor exista întinderi care nu sunt necesare [17].

4.1.3 "RayCast"

"RayCast" reprezintă o metodă comună de identificare al unui obiect în Unity, fiind necesară în selectarea unui obiect. Acesta funcționează prin emiterea unui "ray" dintr-un punct de origine dat, în direcția și distanța necesară. De asemenea pot fi folosit în ignorarea unui obiect dintr-un strat și să ignore sau nu obiecte ale căror coliziuni sunt folosite ca declanșatoare. Pentru a folosi "RayCast" este absolut necesar ca obiectul să aibă implementată componenta din Unity de coliziuni pentru a fi detectată [18].

4.1.4 "RigidBody"

Este o componentă care în momentul în care este aplicată pe un obiect, acel obiect va fi controlat prin motorul de fizică din Unity. Fără script-uri suplimentare, un obiect cu "RigidBody" va fi tras de gravitație și va reacționa cu obiectele din jur numai dacă au același tip de coliziuni. Aflându-se sub controlul motorului de fizică, este recomandat să se folosească funcția "FixedUpdate()", deoarece modificările de fizică sunt realizate în pași de timp care nu coincid cu actualizările de cadru [19].

4.1.5 "Unity URDF Importer"

"Unity URDF Importer" reprezintă un pachet pentru Unity ce permite importarea unui robot definit prin formatul URDF într-o scenă din Unity. Importatorul pasează un fișier URDF și îl importă în Unity folosind "articulation body" [20].

4.2 Biblioteci și clase folosite

4.2.1 Biblioteci folosite în Unity

Pentru partea de script-uri au fost folosite următoarele biblioteci:

- "UnityEngine" care este biblioteca principală din Unity prin care se pot accesa funcțiile și componentele din Unity
- "UnityEngine.InputSystem" este biblioteca prin care se poate implementa cea mai recentă versiune de control din partea utilizatorului provenite de la: tastatură, mouse sau manete
- "UnityEngine.UI" permite lucrul cu elementele vizuale din scenă cum sunt butoanele
- "Unity.Robotics.UrdfImporter.Control" este o bibliotecă provenită din pachetul URDF destinat pentru Unity prin care se poate accesa partea de control al unui braț dintr-un fișier URDF
- "System.IO" este o bibliotecă ce permite lucrul cu fișiere de tip JSON, CSV sau altele
- "TMP" prin aceasta se poate lucra cu componenta TMP din Unity
- "MQTTnet" permite implementarea protocolului MQTT și anume partea de creare sau conexiune la un broker și partea de publicare sau abonare la un topic
- "MQTTnet.Client" prin aceasta se poate crea un client MQTT care să se conecteze la un broker
- "System.Threading.Tasks" este biblioteca care permite implementarea operații asincrone și paralele folosind fire de execuție

Din "System.Threading.Tasks" cea mai utilizată funcționalitate este directiva de "async" prin care compilatorul transforma funcția cu această directivă într-o mașină de stări internă, permițând divizarea și suspendarea ei. Această directivă este folosită împreună cu instrucțiunea "await" ce reprezintă un flag ce marchează momentul de suspendare, moment în care firul de execuție este eliberat pentru alte sarcini și după finalizarea operației așteptate restul codului este reluat automat. Această pereche este utilă deoarece aplicația se execută încontinuu indiferent ce se întâmplă în procesul de comunicație. De asemenea oferă și accesul la clasa "Task" ce reprezintă o operațiune asincronă prin care se poate controla și monitoriza acest tip de cod.

4.3 Clasa "Monobehaviour"

Această clasă aparține bibliotecii "UnityEngine" și este folosită de majoritatea script-urilor dezvoltate pentru Unity. Clasa permite atașarea script-ului de un obiect din scenă și de asemenea oferă următoarele funcții:

- "Update()" care este continuu la fiecare cadru, folosită la implementări unde este necesară o execuție în timp real al codului numai dacă clasa scriptul este derivată din clasa "MonoBehavior"
- "FixedUpdate()" care este apelată de obicei la un interval de 0.02 secunde și este folosită pentru a se asigura că calculele legate de fizică rămân consistente
- "Start()" care este apelată la începutul execuției și este folosită la instanțieri sau la folosirea de elemente statice
- "OnEnable()" este folosită pentru inițializare dinamică sistemelor de control sau alte sisteme moment când acestea devin active
- "OnDisable()" este folosită pentru a dezactiva sistemul de control în momentul când sistemul nu mai este activ sau când este distrus prevenind astfel acțiunile accidentale și scurgerile de memorie

4.4 Biblioteci braț robotic real

Brațul real fiind implementat folosind limbajul de programare micro-python au fost folosite următoarele biblioteci:

- "network" care-i permite plăcii de rețea a microcontrolerului să se conecteze la o rețea WI-FI
- "umqttsimple" prin care se poate implementa protocolul MQTT pe microcontroler
- "espmx" ce conține implementările brațului realizate de către producătorul brațului cum ar fi: setarea poziției brațului, controlul servomotoarelor, citirii și verificării poziției
- "SunctionNozzle" prin care se poate controla pompa oferită de către producător
- "Buzzer" prin care se poate folosi buzzer-ul oferită de către producător

4.5 Fusion 360

Fusion 360 reprezintă o platformă care combină CAD, CAM, CAE, PCB, gestionarea datelor într-o singură și integrată platformă software în cloud [21].

Fusion este capabil să:

- De explorarea ale iterărilor proiectului cu ușurință folosind unelte de modelare 3D [21]
- Să producă părți de mașini CNC la o calitate înaltă cu uneltele de CAM/CAM integrate [21]
- Să acceseze proiecte electronice întregi [21]
- Oferă unelte de simulare 3D pentru proiect [21]
- Se poate explora rezultate care sunt pregătite pentru producție cu proiect generativ [21]

4.6 Ngrok

Ngrok este o aplicație care poate fi folosită între-platforme diferite care creează tuneluri de la un "endpoint" public către un serviciu web care rulează local, permițând dezvoltatorilor să-și expună serverele locale internetului. Integrează toate componentele esențiale cum sunt: "API Gateways", "reverse proxy", "load balancer", "DDoS Protection" care livrează trafic de la serviciul local la internet. [22].

5.1 MaxArm

MaxArm este un brat robotic dezvoltat de compania Hiwonder. Este un brat robotic open-source care este controlat de un microcontroler ESP32. Poate fi folosit în scop educativ sau în proiecte avansate. Robotul este echipat cu servo motoare cu magistrală HTS-35H de înaltă calitate și o pompă de sucțiune [23]. Aceste servomotoare sunt controlate prin comenzi provenite de la portul serial, fiind necesar setarea parametrilor servovului și ID înainte să poată fi folosit. Acest servomotor folosește interfața UART pe "half-duplex", astfel încât terminalul pentru semnal să poată să transmită și să recepționeze semnale.

Brațul prezintă un mecanism articulat specific folosit în construcția de mașini, pentru care necesită utilizarea unor servomotoare cu ax unic, capabile de generarea unui cuplu puternic și să ofere o capacitate de poziționare de înaltă precizie, aceste motoare îndeplinind aceste condiții [24]

Specificații motoarelor HTS-35H [24]:

- Tensiune de lucru: 9-12V DC
- Viteză de rotație 0.18s/60° (DC 11.1V)
- Cuplu 35kg.cm (DC 11.1V)
- Cuplu static maxim 35kg.cm (DC 11.1V)
- Intervalul de rotație: (0 – 240)°
- Curent la mers în gol: 100mA
- Curent de blocaj: 3A
- Acuratețe motor: (0.2)°
- Interval de control al unghiurilor: 0-1000 corespunzând cu 0 – 240°
- Interval de ID-uri: 0-253
- Protecții: Evitarea stărilor de blocaj și supraîncălzire
- Parametrii de feedback: Temperatură, tensiune, poziție
- Mod de lucru: Mod de servo și în mod de motor cu rotație continuă
- Tip angrenaj: Metalic
- Greutate 64g
- Dimensiuni: 54.38*20.14*45.5 mm (lungime*lățime*înălțime)

Prezintă 4 grade de libertate: stânga-dreapta, sus-jos, față-spate, și rotația de obiecte. Prin aceste grade și tehnologia de cinematică inversă, acesta poate să execute o gamă largă de sarcini cum ar fi: apucare, sortare, transportare și stivuire de obiecte. Acesta suportă comunicare prin Bluetooth și WIFI și programare folosind Python prin Micro-Python și Arduino prin C/C++ [23].

Specificații [23]:

- Greutate: 1.3Kg

- Metal și placă din fibră de sticlă
- Grade de libertate: 4
- Alimentare: 12V 5A DC
- Controler: ESP32
- Metode de comunicație: WIFI și Bluetooth
- Motoare: HTS-35H și LFD-01M
- Dimensiuni: 158*160*260 mm

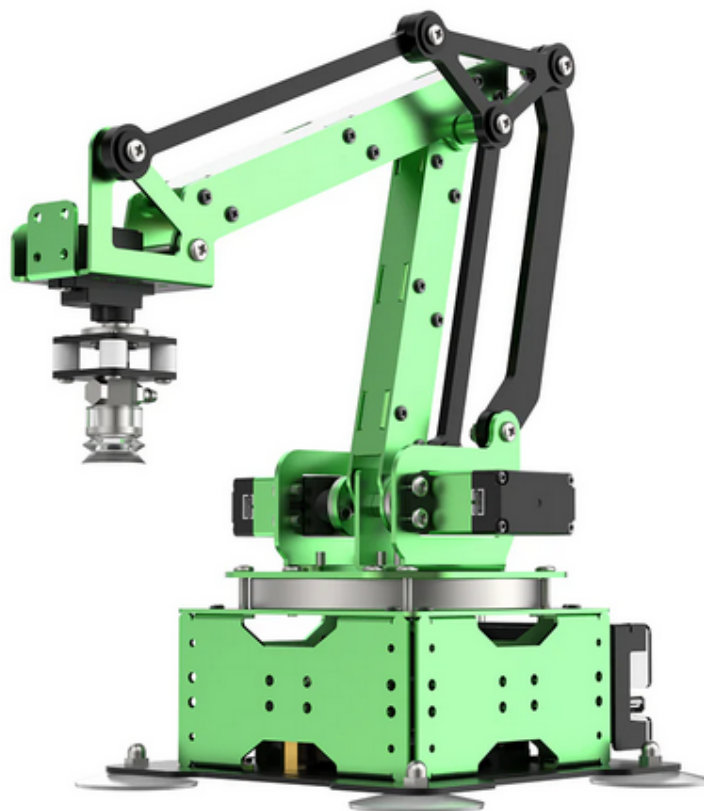


Figura 1: Brațul Robotic MaxArm

5.2 Oculus Quest 3S

Oculus Quest 3S sunt o pereche de ochelari de realitate mixtă de buget dezvoltată de compania Meta [25].

Specificații [25]:

- Procesor: Snapdragon XR2 Gen 2
- Camere: 2 camere RGB cu PPD de 18
- Stocare: 128GB/256GB
- DRAM: 8GB
- Rezoluție ecran: 1832x1920 pixeli/ochi cu 20 PPD și 773 PPI
- Rată de reîmprospătare: 72Hz, 90Hz, 120Hz
- Unghi de vizualizare: 96 de grade orizontal, 90 de grade vertical
- Optică: lentile fresnel
- Conectivitate: USB-c și WIFI
- Durată baterie: aproximativ 2.6 ore
- Timp de încărcare: aproximativ 1.9 ore pe 18Watts



Figura 2: Ochelari de realitate virtuală Oculus Quest3S

Arhitectura sistemului este una distribuită integrând concepte de IoT. Aceasta structură are ca scop asigurarea unui control determinist și în timp real al unui braț robotic fizic prin intermediul unui mediu de realitate virtuală.

Sistemul a fost realizat din 4 niveluri funcționale principale, care comunică pentru a transforma controlul provenit de la utilizator în mișcarea mecanică a brațului real.

6.1 Nivelul de simulare și interacțiune

Acest nivel rulează folosind casca de realitate virtuală cu aplicația efectivă rulând pe calculatorul principal, fiind dezvoltată folosind motorul grafic Unity 3D. Rolul său este de a prelua comenzile provenite de la utilizator, de a randa mediul 3D și de a procesa matematica și controlul asociat mișcării. Componentele de bază ale acestui nivel sunt:

- **Subsistemul VR și Gestionarea comenzilor:** Implementat folosind pachetul "XR Interaction Toolkit". Aceasta interfațează direct cu driverele căștii pentru a citi poziția spațială pentru cap și manete. Scriptul "ToggleXRControls" gestionează stările de control, în timp ce maparea butoanelor fizice este realizată prin asocieri în profilele de intrări prin "XRI Default Input Actions".
- **Modelul cinematic și "Digital Twin":** Reprezentarea virtuală a robotului se bazează pe importul fișierului de descriere URDF. Acesta definește ierarhia rigidă a componentelor și a tipului articulațiilor. Geometria vizuală este asigurată prin randarea 3D a fișierelor de tip STL.
- **Controlul direct:** Reprezintă o modalitate de mișcare a articulațiilor în care comanda care provine de la utilizator se aplică direct pe articulațiile principale ale brațului.
- **Modulul de cinematică inversă:** Mișcarea robotului virtual provine de la script-ul de cinematică inversă și cel de control al efectorului final. Atunci când utilizatorul folosește maneta, scriptul de control al efectorului final actualizează poziția unui obiect din scenă. Script-ul de cinematică inversă preia coordonatele carteziene X, Y, Z ale țintei și calculează unghiurile de rotație necesare pentru fiecare articulație.
- **Elemente ajutătoare de control:** Implementat prin citirea de JSON al fișierului care conține lista de controale și acțiunile pe care utilizatorul le poate face asupra robotului. Conține și indicatoare pentru a marca dacă acțiunea respectivă a fost realizată.

6.2 Nivelul de comunicație

Acest nivel asigură legătura bidirecțională wireless între Unity și robotul fizic. Componentele de bază ale acestui nivel:

- **Canalul MQTT:** Pentru transmiterea comenzilor, a fost implementat protocolul MQTT ce folosește ca model o arhitectură de tip "Publish/Subscribe".
- **Tunelul prin "ngrok"** prin care se poate realiza o conexiune din orice rețea între Unity și ESP.
- **Fluxul de comandă dintre Unity și ESP:** Este folosit un script care preia unghiurile de la brațul virtual, îl serializează într-un pachet de date și le publică pe brokerul MQTT sub un topic dedicat.
- **Fluxul de Feedback dintre ESP și Unity:** Este folosit un script în Unity ce ascultă pe topicuri ce conțin mesaje provenite de la robotul real legate de starea curentă și confirmări de execuție ale mișcării.

6.3 Nivelul de procesare al robotului fizic

Sistemul de procesare este amplasat pe șasiul robotului și care folosește un microcontroler ESP32, programat folosind limbajul de programare Micropython. Componentele de bază ale acestui nivel:

- **Inițializarea și gestiunea rețelei:** La inițializarea sistemului, este apelat un script ce se ocupă de configurația rețelei microcontrolerului prin conectarea acestuia la un punct de acces local și pentru atribuirea unei adrese IP statică. Pe urmă utilizând o implementare pe baza protocolului MQTT se stabilește o sesiune TCP cu brokerul local.
- **Gestionatorul datelor și parsarea datelor:** Scriptul "main" rulează bucla de execuție a procesorului. Acesta comunică direct cu script-ul de gestionare MQTT al microcontrolerului. De fiecare dată când brokerul MQTT livrează un pachet trimis de Unity, script-ul de gestionare MQTT interceptează pachetele, se ocupă de deserializarea acestora și convertește valorile text în variabile care să fie trimise către motoare

6.4 Nivelul de execuție hardware

Acest nivel reprezintă interfața fizică a sistemului, unde semnalele digitale procesate de ESP sunt convertite în semnale mecanice și feedback local. Pentru implementarea acestui nivel producătorul a oferit o serie de scripturi și anume:

- **Pentru controlul servomotoarelor seriale:** Brațul robotic real folosește servomotoare de magistrală conectate în topologie de tip lanț, printr-un fir de comunicație UART. Scriptul conține logica de comunicare cu controllerul intern al motoarelor. Pentru fiecare mișcare, scriptul generează un cadru binar de date în format hexazecimal care conține: un header de sincronizare, ID-ul unic al motorului, poziția unghiulară a motorului convertită la valori între 0 și 1000, timpul alocat mișcării și un octet de paritate pentru detecția erorilor de transmisie serială.
- **Controlul motoarelor prin PWM:** Pentru mișcări auxiliare, există un script care generează semnale de tip PWM utilizând timer-ele hardware ale ESP-ului prin ajustarea factorului de umplere.
- **Sistemul de prindere prin vacuum:** Efectorul final al brațului folosește o pompă de vid pentru manipularea obiectelor. Script-ul robotului ce se ocupă de pompă mapează semnalul boolean primit din Unity. La primirea valorii logice de "1", scriptul activează pompa.
- **Sistemul de feedback și diagnostic local:** Producătorul a oferit posibilitatea de implementa un asemenea sistem direct pe placa ESP prin folosirea unui buzzer și a unui led.

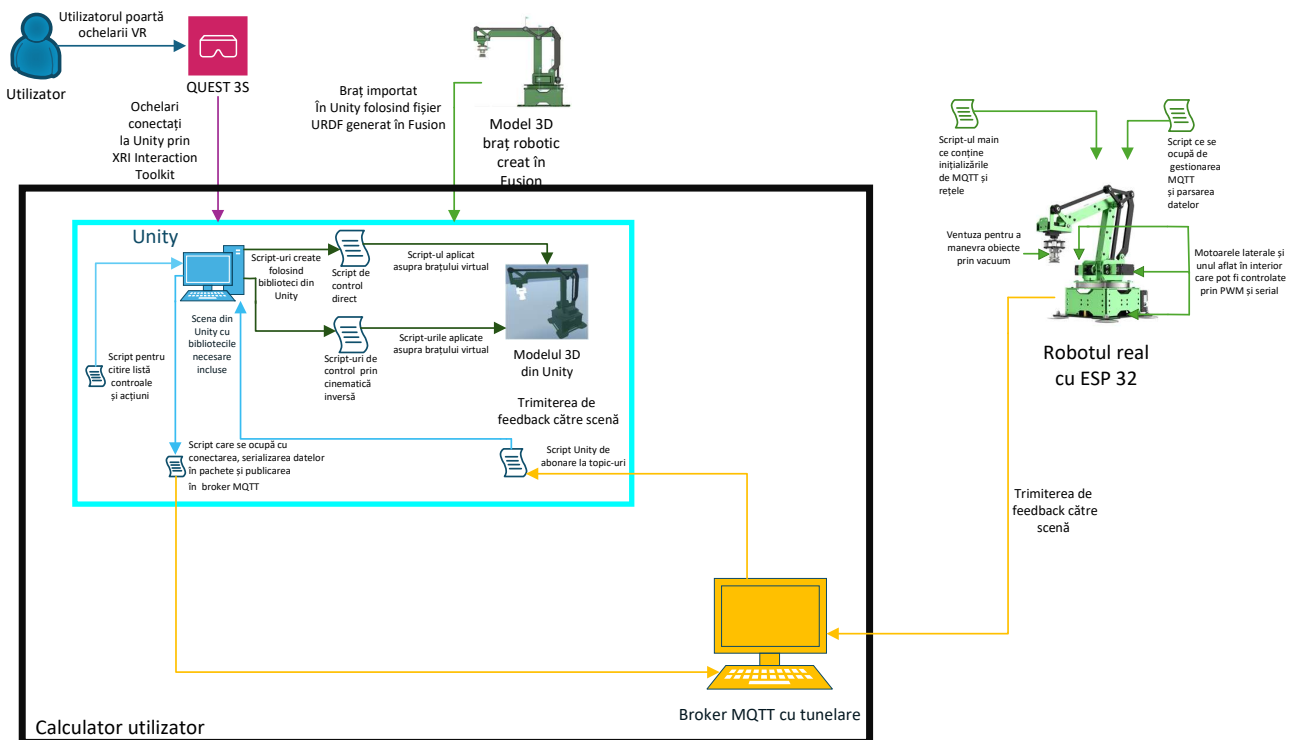


Figura 3: Arhitectura sistemului

7.1 Realizarea modelului 3D al brațului robotic

Pentru a putea implementa brațul robotic în Unity a fost necesară realizarea modelului 3D. Pentru a realiza modelul 3D a fost necesară împărțirea brațului robotic pe componente individuale și măsurarea acestora. Astfel modelul a fost împărțit în 11 părți, având următoarele dimensiuni aproximative din lipsa de instrumente de măsurat de precizie exprimate în milimetri:

- Baza principală robotului: 140 x 140 x 66

Aceasta reprezintă baza robotului de care este prinsă componenta de rotație stânga-dreapta pe motorul intern care mai conține și pompa prin care produce vacuumul pentru suțiu

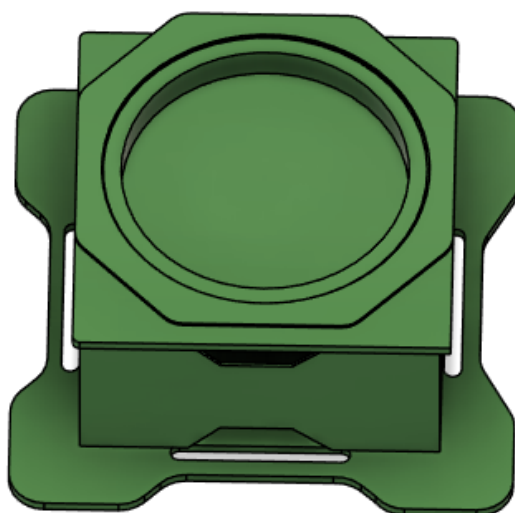


Figura 4: Modelul 3D al bazei brațului

- Baza secundară: 109 x 85 x 63

Este componenta care se rotește stânga-dreapta atașată de motorul din bază și de care sunt atașate cele 2 motoare de pe o parte și alta care-i permit brațului să deplaseze sus-jos față spate, și de care mai sunt atașate tijele de susținere verticală și brațul vertical principal.

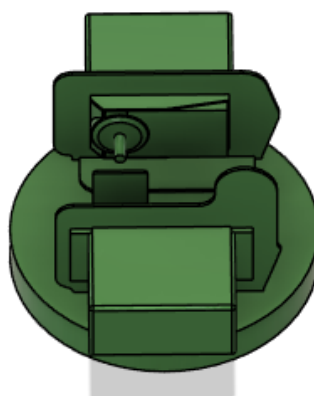


Figura 5: Modelul 3D al bazei secundare

■ Segment vertical principal: 20 x 15 x 150

Acesta este prins de motorul din stânga de pe zona rotativă prin care brațul se poate deplasa față-spate. Acesta funcționează și ca susținere pentru brațul orizontal principal.

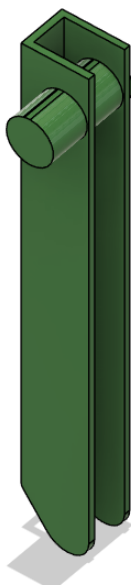


Figura 6: Modelul 3D al segmentului vertical principal

■ Segment orizontal principal: 180 x 25 x 25

Este prinsă de segmentul vertical principal și tija ajutătoare verticală 1 și de aceasta se ocupă în principal de susținerea suportului pentru pompă.

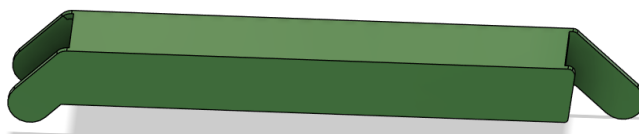


Figura 7: Modelul 3D al segmentului orizontal principal

■ Segment lateral de urcare-coborâre: 67 x 2 x 20

Acesta este atașat la motorul drept de pe zona rotativă și prin intermediul acestuia se poate deplasa brațul sus-jos.

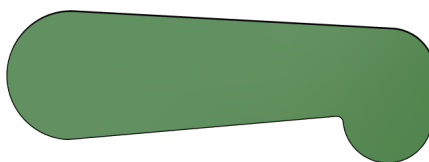


Figura 8: Modelul 3D al segmentului lateral de urcare-coborâre

■ Tijă ajutătoare verticală 1: 5 x 3 x 140

Este prinsă între segmentul lateral de urcare-coborâre și segmentul orizontal principal și are ca rol să ajute la urcarea sau coborârea segmentului orizontal principal și de asemenea la susținerea brațului când trebuie să se deplaseze prin segmentul vertical principal față-spate.

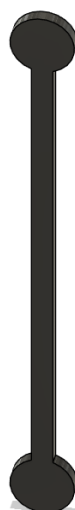


Figura 9: Modelul 3D al tijei ajutătoare verticală 1

■ Tijă ajutătoare verticală 2: 15 x 5 x 144

Este prinsă între zona rotativă și tija ajutătoare orizontală 2 și are ca rol să ofere robotului o stabilitate mai bună atât când se deplasează față-spate cât și când stă nemișcat.



Figura 10: Modelul 3D al tijei ajutătoare verticală 2

■ Tijă ajutătoare orizontală 1: 146 x 5 x 5

Este prinsă între tija ajutătoare orizontală 2 și suportul pentru pompa de sucțiune și are ca rol să ajute în susținerea și balansul suportului pentru pompă.



Figura 11: Modelul 3D al tijei ajutătoare orizontală 1

■ Tijă ajutătoare orizontală 2: 84 x 5 x 37

Este prinsă între tija ajutătoare 1, tija verticală de susținere 2 și segmentul vertical principal și are ca rol să susțină tija ajutătoare 1 să se deplaseze după segmentul orizontal principal și să ajute tija verticală de susținere 2 să se deplaseze după segmentul vertical principal.

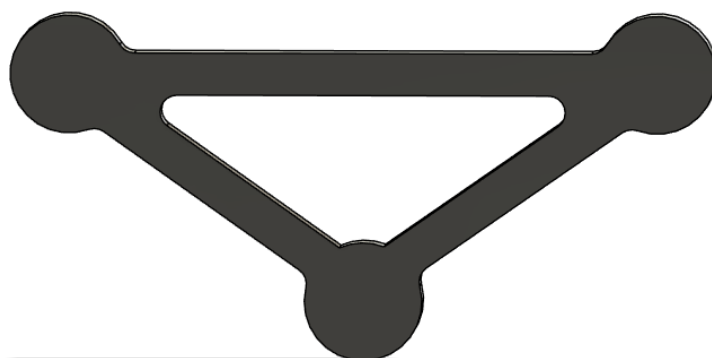


Figura 12: Modelul 3D al tijei ajutătoare orizontală 2

- Suportul ansamblului pentru ventuza de sucțiune : 42 x 1 x 44 Este prinsă de segmentul orizontal principal și de tija orizontală ajutătoare 2 și are ca rol să susțină ansamblul pentru ventuza de sucțiune cât și mini motorul.

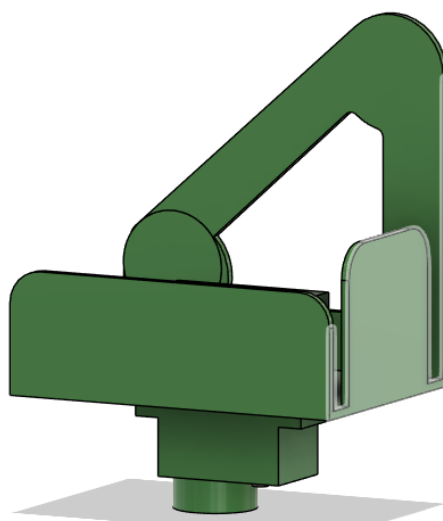


Figura 13: Modelul 3D al suportului pompei de sucțiune

- Ansamblul pentru ventuza de sucțiune : 26 x 26 x 35
Acesta este prinsă de suport și este conectată printr-un furtun la pompa din interiorul bazei principale și este componenta prin care brațul robotic poate să manipuleze obiecte.

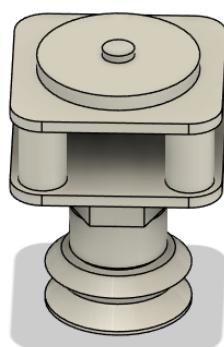


Figura 14: Modelul 3D al ansamblului pentru ventuza de sucțiune

După modelarea individuală a componentelor a urmat prinderea acestora între ele. Pentru a le putea uni s-a folosit "Joint" din Fusion. "Joint" permite prinderea componentelor între ele și stabilirea tipului și sensului/unghiului de rotație/deplasare. Principalul "joint" folosit pentru prindere și stabilire unghiului de rotație este cel de tip "revolute" care permite rotații de până la 360 de grade. Vizual aceste "joints" sunt reprezentate sub formă de cerc ce are prins un steguleț, steguleț ce reprezintă sensul pozitiv/negativ de rotație al componentei.

Prinderea între 2 componente în Fusion a fost realizată folosind numai un singur "joint" între componenta care trebuie prinsă și componenta de care se prinde.

Prinderea folosind un singur "joint" a fost necesară deoarece dacă se folosesc 2 "joints" pentru a se prinde o componentă între altele 2 apar conflicte între "joints" ceea ce ar face componenta prinsă să nu se mai miște, neștiind de care este controlată.

Acest lucru a mai fost necesar pentru a transforma dintr-o structură de paralelogram așa cum este robotul real, într-o structură de arbore. Structura de arbore este necesară pentru a putea genera într-un mod complet fișierul URDF al robotului, folosit pentru a-l putea importa în Unity.

Generarea fișierului URDF a fost realizată folosind un script de conversie care analizează modelul 3D de la bază la ultima componenta atașată și invers, și în funcție de "joints" care sunt prinse generează:

- partea de "link" care se ocupă de gestionarea părții vizuale ale componentelor
- partea de "joints" care se ocupă de partea de mișcare ale componentelor

Modelul final al robotului după ce toate componentele au fost prinse și unghiurile de rotație au fost setate:

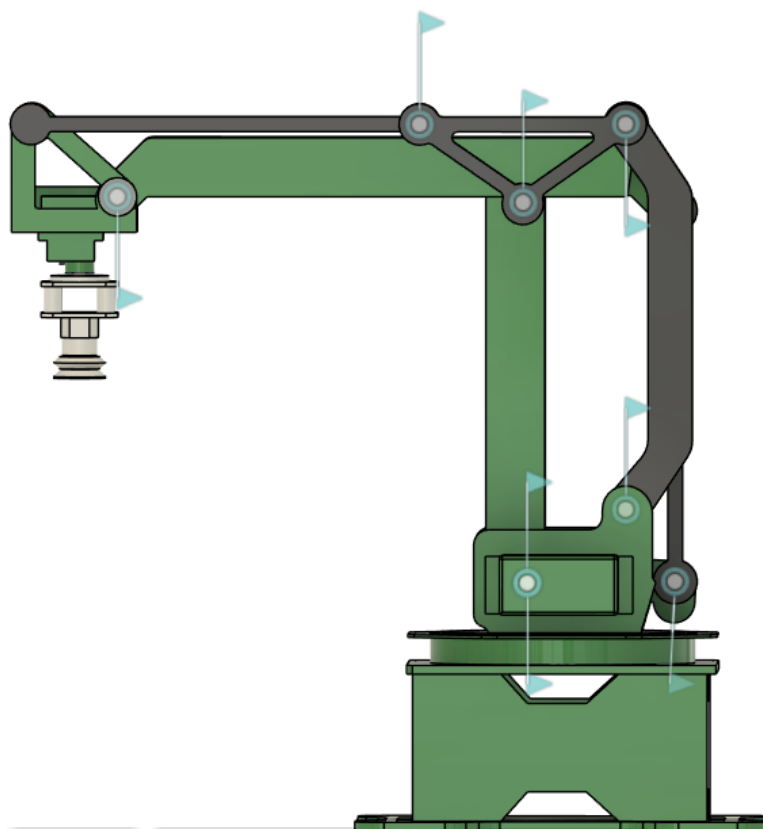


Figura 15: Modelul 3D al brațului MaxArm

După generarea fișierului URDF, acesta a fost importat în Unity. Însă până la importul în Unity a fost necesar un mediu de lucru cu care se poate interacționa în VR.

7.2 Mediul din Unity

Pentru mediul de lucru în VR s-a folosit ca bază un mediu demo oferit de cei de la Unity, care oferă minimul necesar pentru implementările ulterioare, acestea fiind: scripturi necesare pentru mișcarea și interacționarea cu mediul, elemente cu care se poate interacționa, un mini tutorial despre cum funcționează mediul, și alte elemente.

Fiind un mediu demo, acestuia i-sa adus câteva modificări necesare pentru al aduce la un stadiu în care se pot realiza implementările ulterioare: dezactivarea manetei drepte, și a unor butoane de pe maneta stângă, fiind ulterior folosite pentru a implementa sistemul de control al brațului, și re-maparea acestora, eliminarea unor elemente de decor.

7.3 Importarea în Unity

Importarea în Unity a fost realizată prin folosirea pachetului URDF care preia structura și ierarhia din fișierul URDF generat anterior, le analizează, o importează în Unity și generează fișierele de textură fiecărei componente.

Pe lângă import și texturi pachetul atribuie fiecărei componente un corp de tip "articulation", scripturile de definire al tipului de "joint", de control al "joints" și de parametri fizici. Problema cu script-ul este că deși generează fișierele de textură, acesta nu aplică automat texturile pe componente, aplicarea acestora realizându-se manual.

Structura ierarhică a componentelor:

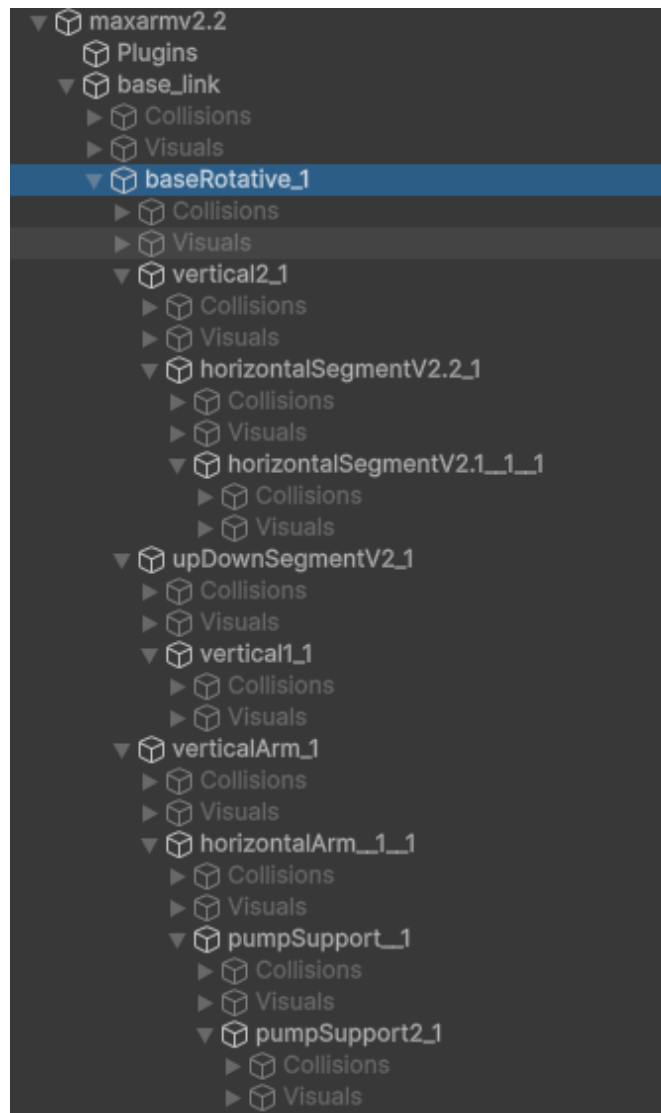


Figura 16: Structura ierarhica în Unity al brațului robotic

Având structura robotului și modelul 3D importat, texturile aplicate și verificarea fiecărei componente să aibă tipul necesar de "body" și script-urile necesare, s-a putut trece la setarea proprietăților brațului și setarea constrângerilor între componente.

7.4 Setarea proprietăților

Setarea proprietăților constă în alegerea valorilor pentru rigiditatea, amortizarea și forța pentru ca componentele să poată fi folosite. Realizarea acestor setări a fost necesară deoarece:

- Rigiditatea reprezintă forța cu care articulația ajunge la țintă. Dacă are valoarea de 0 sau valori mici de exemplu sub 10000, atunci articulația nu se mișcă, sau face mișcări lente.
- Amortizarea reprezintă proprietatea care ajută la diminuarea/oprirea tremurațiilor brațului. Dacă are valoarea de 0 sau valori mici, de exemplu sub 100, atunci la fiecare mișcare a articulației aceasta ar începe să tremure ducând la instabilitate și probleme la fizica robotului, cum ar fi desprinderea și împrăștierea componentelor brațului.
- Forța pe care motoarele/articulațiile o au nevoie pentru ale putea acționa. La valori mici, de exemplu sub 1000 motoarele brațului n-ar avea forța necesară să se miște și în consecință nu vor putea fi folosite.

Având în vedere cele mai de sus au fost alese următoarele valori pentru proprietăți:

- Rigiditatea 100000
- Amortizarea 1000
- Forța 10000

Pentru aplicarea acestor valori există 2 abordări principale: Prima abordare constă în aplicarea acestor valori direct în proprietățile articulației. Însă pentru a folosi această abordare există o problemă: valorile puteau să fie aplicate numai în momentul în care scena se afla în modul de "play". După ieșirea din modul de "play" valorile reveneau la cele inițiale. Astfel de fiecare dată când trebuia testată mișcarea unei componente sau ale unui set de componente trebuia aplicate valorile de fiecare dată.

A doua abordare a fost și cea aleasă și folosită în implementare și constă în crearea și folosirea unui script separat pentru a putea suprascrise mereu valorile. Astfel chiar dacă scena se află în modul de lucru, acest script se execută ultimul suprascrivând valorile din secțiunea de proprietăți ale articulației.

Acest script folosește următoarele funcții: "Drive()", "ApplySettings()" și "Start()".

Funcția "Drive()" primește ca parametrii un obiect de tip "ArticulationBody", și proprietățile de rigiditate, amortizare și forță. Acestea sunt accesate și aplicate folosind componenta din Unity de "xDrive".

Funcția "ApplySettings()", care apelează funcția de "Drive()" care este aplicată pe fiecare componentă în parte.

Funcția "Start()" este apelată funcția de "ApplySettings()" cu o întârziere de 0.1 secunde asigurând astfel că proprietățile sunt mereu suprascrise.

7.4.1 Compensări cinematice

Compensările cinematice reprezintă condiționarea mișcării unor componente față de alte componente.

În mediul din VR compensările au fost necesare pentru că față de robotul real în care dacă un segment principal se deplasa, și tijele de susținere se deplasau o dată cu el când se dădea comanda de deplasare. Fără aceste compensări tijele față de segmentele principale rămâneau fixe. În cazul susținerii pompei, acesta fiind un copil al segmentului orizontal principal, acesta îi copia unghiul la care se afla.

Compensările cinematice realizate pot fi grupate în 2 categorii:

- simple, în care o componentă are unghiul de deplasare în același sens de componenta care îi condiționează mișcarea sau în sens opus acesteia
- compuse, în care unghiul de deplasare al unei componente este condiționate de minim 2 componente

Pentru a crea compensările cinematice a fost creat un script, care are implementat funcțiile: "NegativeCompensation()", "PositiveCompensation()", "FixedUpdate()".

Funcția de "NegativeCompensation()" preia ca parametrii un "ArticulationBody" care reprezintă componenta părinte care dă unghiul de deplasare și un "ArticulationBody" care reprezintă componenta copil pe care este aplicat unghiul de la părinte. Se folosește de unghiul componentei părinte exprimată în grade, pe care o aplică cu semn opus asupra unghiului la care trebuie să ajungă componenta copil. Pentru a seta unghiul componentei copil este necesară accesarea componentei "target" care se află în categoria de "Drive" a lui "ArticulationBody"

Funcția de "PositiveCompensation()" care preia ca parametrii un "ArticulationBody" care reprezintă componenta părinte care dă unghiul de deplasare și un "ArticulationBody" care reprezintă componenta copil pe care este aplicat unghiul de la părinte. Funcția folosește de unghiul componentei părinte exprimată în grade, pe care o aplică cu același semn asupra unghiului la care trebuie să ajungă componenta copil. Pentru a seta unghiul componentei copil este necesară accesarea componentei "target" care se află în categoria de "Drive" a lui "ArticulationBody"

"FixedUpdate()" reprezintă funcția principală a script-ului. În această funcție sunt apelate funcțiile de mai sus, și sunt realizate restul calculelor pentru componentele ce necesită compensare față de 2 sau mai multe componente.

Compensările au fost grupate în 2 categorii principale: pentru tije de susținere și pentru segmentele principale.

Pentru tije compensările au fost realizate astfel:

- "PositiveCompensation(segmentul vertical principal, tija verticală de susținere 2)"
- "PositiveCompensation(segmentul vertical principal, tija orizontală de susținere 2)"
- "NegativeCompensation(segmentul orizontal principal, tija orizontală de susținere 1)"
- tija verticală de susținere 1 se va deplasa în sens opus cu suma unghiurilor segmentul lateral și a segmentului vertical principal

În cazul unor segmente principale a fost necesară setarea de constrângeri. Pentru segmentul orizontal principal deoarece când se deplasa segmentul vertical principal, acesta îi copia unghiul de deplasare, fiind necesar ca segmentul orizontal principal să-și păstreze poziția orizontală pe toată durata mișcării segmentul vertical principal. Tot în categoria segmentelor principale a fost necesară și setarea de constrângeri pe suportul de pompă deoarece indiferent de sensul de deplasare al segmentelor principale, acesta trebuie să rămână perfect paralelă cu solul.

Astfel pentru segmentul orizontal principal și suportul pompei, au fost setate următoarele constrângeri:

- segmentul orizontal principal se va deplasa în sens opus cu suma unghiurilor segmentului vertical principal și segmentul care deplasează segmentul orizontal principal pe axa Y
- suportul pompei are următoarea constrângere:

$$\begin{aligned} \text{suportPompei} = & (2 * \text{unghiulTijeOriizontale1}) \\ & - \text{unghiulSegmentuluiOrientalPrincipal} \\ & - \text{unghiulSegmentuluiVerticalPrincipal} \end{aligned}$$

7.5 Sistemul de control al brațului în Unity

Sistemul de control în Unity reprezintă modul în care comenzile provenite de la manetele câștii controlează componentele care sunt direct conectate la motoare. Fără acest sistem de control, robotul din Unity ar fi fost doar de decor. În implementarea sistemului de control au fost folosite 2 metode de implementare. Prima metodă se bazează pe controlul direct al motoarelor folosind butoanele de pe manete, cuprinzând și partea de resetare a poziției robotului, cât și funcționalitatea de manevrare a obiectelor. A doua metodă este mai utilizată și se bazează pe un algoritm de cinematică inversă și pe un efector final.

7.5.1 Metoda de control bazată pe control direct

Metoda aceasta este cea mai simplă de implementat și cu care se pot face teste rapid și se bazează pe atribuirea modului de deplasare al componentelor părinte direct la butoanele manetelor. Pentru implementare s-a creat un script prin care se face această mapare.

Script-ul la bază implementează 2 funcții principale: "OnEnable()", "Update()".

Pentru controlul efectiv a fost necesar folosirea proprietății "direction" a "joint-urilor", fiind accesată folosit componenta de "JointControl".

Pentru controlul bazei rotative, sunt citite valorile de pe axa X a "thumbstick-ului", fiind setate praguri datorită sensibilității, acesta fiind foarte sensibil la atingere. Pentru rotirea spre dreapta dacă s-a depășit acel prag, direcția de deplasare va fi negativă, iar pentru stângă, dacă s-a depășit pragul, direcția de deplasare va fi pozitivă.

Pentru segmentul vertical principal, sunt folosite cele 2 "triggere" de pe manetă. Dacă se detectează o apăsare pe "trigger-ul" frontal atunci, sensul de deplasare este spre direcția negativă a axei Z, adică în față, iar dacă pentru "trigger-ul" lateral este detectată o apăsare atunci sensul de deplasare este pe partea pozitivă a axei Z, adică în spate.

Pentru segmentul lateral de urcare-coborâre au fost folosite cele 2 butoane marcate cu literele de pe manetă. Coborârea pompei se realizează pe direcția pozitivă a axei Y, la apăsarea butonului principal, iar urcarea pompei este realizează pe direcția negativă a axei Y, la apăsarea butonului secundar.

7.5.2 Metoda de control bazată pe cinematică inversă

Această metodă de control a unui braț robotic/robot este cea mai utilizată și se bazează pe implementarea unui algoritm ce rezolvă cinematica inversă a acestuia. Algoritmul implementat pentru braț se bazează pe geometrie, trigonometrie și se folosește de legea cosinusului. Implementarea a fost realizată folosind 2 funcții într-un script dedicat implementării. Prima funcție de "FixedUpdate()" este cea în care conține pașii de dinaintea implementării algoritmului și implementarea propriu-zisă. Cea de a 2 a funcție "SetJointAngle()" se ocupă de setarea unghiului "joint-urilor", din componenta de "xDrive" a unui "ArticulationBody".

În funcția "FixedUpdate()" pașii de dinaintea implementării a constată în setarea unor constate ce reprezintă lungimea segmentelor ale robotului și scalarea pe axele X, Y, Z a elementelor componente în raport cu dimensiunea la care se află robotul în scenă.

Implementarea propriu-zisă a algoritmului este realizată prin a reducerea problemei cinematicii inverse din planul 3D la 2D, formând un triunghi dreptunghic, ajutând la simplificarea calculelor ulterioare. Reducerea orizontală a fost realizată prin calculul distanței de la centrul bazei până la țintă pe orizontală, de unde s-a scăzut "offset" pentru segmentul de deplasare stânga-dreapta, acesta nefiind așezat perfect în centrul axului care rotește brațul stânga-dreapta, reducerea fiind notată cu r și fiind implementată după formula:

$$r = \sqrt{x^2 + y^2} - L_1 \quad (1)$$

unde x și y reprezintă coordonatele în planul orizontal, și L_1 reprezintă un "offset" pentru segmentul de deplasare stânga-dreapta, acesta nefiind așezat perfect în centrul axului care rotește brațul stânga-dreapta

Reducerea pe verticală a fost realizată prin scăderea din valoarea de pe axa Z a înălțimii bazei fixe, reducerea fiind notată cu h și fiind implementată după formula:

$$r = z - L_0 \quad (2)$$

unde z reprezintă coordonata în plan vertical, și L_0 reprezintă înălțimea bazei fixe

După realizarea calculelor, pe baza reducerilor a fost calculată distanța totală directă pentru țintă folosind reducerile calculate:

$$d = \sqrt{r^2 + h^2} \quad (3)$$

Au fost implementate 2 limite necesare: o limită în cazul în care brațul este complet întins, și cealaltă în cazul în care ținta este prea aproape de baza brațului, ceea ce ar duce la pliere și posibilă la desprinderea componentelor. Pentru limite s-a introdus un factor de scalare s care se calculează ca suma dintre 2 laturi împărțite la distanța totală calculată. În cazul limitei de pliere a brațului, se recalculează factorul s ca raport dintre valoarea absolută, diferența celor 2 laturi la care s-a adunat o valoare de siguranță și distanța totală. În cazul în care se ating limitele r , h sunt înmulțite cu factorul s , moment în care coordonatele țintei se ajustează proporțional cu factorul de depășire s , făcând ca brațul să rămână perfect întins și să nu se plieze peste limita admisă.

După setarea limitelor și pe baza calculelor de mai sus au fost calculate cosinusurile unghiurilor interioare pentru segmentul vertical principal și cel lateral de urcare-coborâre folosind legea cosinusului:

$$\cos A = \frac{L_2^2 + L_3^2 - d_2}{2 * L_2 * L_3} \quad (4)$$

și

$$\cos B = \frac{L_2^2 + d_2 - L_3^2}{2 * L_2 * d} \quad (5)$$

unde,

laturile L_2 și L_3 reprezintă tija de susținere prinsă între segmentul lateral, cel orizontal și vertical principal și d distanța totală

Pentru finalizarea calculelor și folosirea valorilor în calculul unghiurilor finale a fost necesară convertirea calculelor pentru cosinus la unghiurile necesare și calcularea unei noi valori ca pompa să fie îndreptată direct în jos. Valoarea pentru pompă s-a notat cu c și s-a calculat folosind formula:

$$c = \arctan(h/r) \quad (6)$$

și pentru convertirea la unghiurile necesare a fost aplicată arccosinus care a fost convertită din radiani în grade, folosind constanta definită în Unity de 57.29578, acestea fiind notate cu a , respectiv b . Astfel unghiurile pentru segmentul lateral și vertical principal au fost calculate prin:

$$unghiSegmentVertical = 90 - a \quad (7)$$

$$unghiSegmentLateral = -90 + (b + c) \quad (8)$$

Valorile de 90 reprezintă gradele în care cele 2 componente stau în starea de repaus inițială, mișcările produse de braț ducând la modificări pozitive sau negative din aceste valori.

Aplicările efective pe "joints" au fost realizate prin actualizări continue ale unghiurilor calculate mai sus folosind funcția de "Mathf.LerpAngle()" în care: se citește unghiul curent la care se află brațul, se verifică unghiul la care trebuie să ajungă segmentul respectiv, se calculează o valoare intermediară și la final este suprascris unghiul vechi cu cel nou calculat.

Pentru completarea implementării trebuie și partea de efector final pe care brațul să o urmeze permițându-i astfel să poată fi controlat. Acest efector poate fi un obiect din scenă sau pompa care sunt controlate prin intermediul manetei. Pentru implementare a fost folosit ca efector un obiect sferic în scenă. Pompa sau suportul de pompă nu a putut fi folosit datorită că sunt copii ale altor componente, și folosirea lor putea duce la efecte ne-așteptate din partea brațului virtual.

Parte de efector final a fost implementată într-un script separat. Scriptul implementează următoarele funcții: "OnEnable()", "OnDisable()" și "Update()".

În "Update()" este definit un vector ce conține 2 coordonatele pentru mișcarea efectorului: un punct pentru axa X și, respectiv axa Y. După definire se verifică dacă cele 2 perechi pentru deplasarea sus-jos, față-spate nu sunt nule. În cazul în care nu sunt nule în funcție de direcția de deplasare se incrementează sau decrementează din axe o valoare bazată pe viteza deplasare a efectorului cu "Time.deltaTime" care permite deplasarea în scenă la o viteză constantă indiferent la câte cadre pe secundă rulează scena. De asemenea tot în această funcție se implementează partea de reset a efectorului la poziția inițială în scenă, poziția fiind dată de un vector ce cuprinde cele 3 coordonate din 3D.

La final se folosește funcția "Mathf.Clamp", care preia poziția curentă a obiectului și o forțează să rămână strict într-un interval de valori.

De asemenea partea de control mai implementează două funcționalități importante: resetul brațului care poate fi folosită indiferent de modul de control și cea în care robotul să poată să manevreze obiecte.

7.6 Resetarea robotului

Resetarea brațului la poziția inițială este realizată printr-un script dedicat. Implementarea se folosește de următoarele funcții: "OnEnable()", "FixedUpdate()" și "ResetPosition()".

Funcția de "ResetPosition()" preia ca argument un "ArticulationBody" căruia îi resetăm poziția. Resetarea poziției este realizată prin setarea proprietății "target" din "xDrive-ul" argumentului funcției la valoarea 0, aceasta fiind valoarea de bază la care se află "joints".

În funcția "FixedUpdate()" sunt realizate apeluri succesive ale "ResetPosition()" pe "ArticulationBodys" părinte folosite în compensări, numai dacă butonul atribuit resetului a fost apăsat.

7.7 Manevrarea obiectelor

Această funcționalitate este cea care permite brațului prin intermediul pompei să ridice obiecte și să-l coboare. Se bazează pe următoarele funcții: "OnEnable()",

, "OnDisable()", "Update()", "PickUpObject()", "DropObject()", "MoveObject()" care se ocupă de parte de deplasare al obiectului.

Această funcționalitate față de celelalte se folosește de componenta de "RayCast" și de "RigidBody", fiind aplicare pe obiectul pe care brațul trebuie să-l manipuleze.

Funcția "PickUpObject()" se ocupă de prinderea obiectelor și preia ca argument un obiectul care trebuie manipulat, verifică dacă este de tipul "RigidBody()" și dacă este de acel tip copiază obiectul de tip "GameObject" într-o altă variabilă de tip "RigidBody()", și pe acea variabilă este făcută următoarea configurație:

- dezactivarea gravitației astfel încât în momentul când obiectul este apucat să nu fie tras de gravitația din Unity
- aplicarea unei constrângeri care prin care obiectul să nu se poată roti
- setarea părintelui obiectului ca fiind pompa
- activarea proprietății de "isKinematic" pentru ca obiectul să poată fi controlat de către sistemul de fizică, respectiv de către braț
- la final se atribuie obiectului global de tip "GameObject" argumentul funcției și implicit setările realizate

Funcția de "DropObject()" care se ocupă de desprinderea obiectului și preia ca argument o variabilă de tip bool folosită ca să marcheze dacă obiectul să-și mențină sau nu poziția în care se află, fiind necesară parte a sistemului de reset al brațului. În cazul în care se activează reset-ul pompa să lase obiectul în ultima poziție în care se afla.

Astfel este folosită o variabilă de tip Vector3 în care se salvează poziția curentă a obiectului în planul 3D, este verificată dacă obiectul curent nu este nul și dacă nu este, este aplicată următoarea configurație :

- reactivează gravitația obiectului
- dezactivează constrângerea
- setează părintele ca fiind nul
- dezactivează proprietatea de "isKinematic"
- obiectul manevrat și obiectul curent de tip "RigidBody" devin nule

De asemenea se verifică valoarea argumentului. Dacă este adevărată atunci: poziția sa devine cea salvată în variabila de tip Vector, viteza liniară și cea unghiulară a obiectului sunt setate la 0. În caz contrar obiectului a ajuns la poziția dorită, și poziția și viteza liniară devin cele ale pompei.

Funcția de "MoveObject()", verifică mai întâi dacă distanța dintre obiect și poziția pompei s-a modificat. Dacă s-a modificat atunci se calculează direcția de deplasare a obiectului, care este de tipul Vector3, ca diferență între poziția pompei și poziția obiectului. La final este calculată forța obiectului necesară să ajungă la destinație ca produs între distanța de deplasare cu un factor de multiplicare ales arbitrar.

În funcția "Update()" se apelează funcțiile implementate mai sus pe baza unor condiții. Pentru funcția "DropObject()", apelul se realizează atunci când: se apasă butonul de reset, moment în care obiectul manevrat își păstrează poziția sau dacă se apasă din nou butonul a fost apăsat a doua oară de la momentul când acesta a fost apăsat pentru a ridica un obiect. Funcția "PickUpObject()" se apelează în momentul în care s-a apăsat butonul pentru prima dată și dacă variabila "holdObject()" nu este nulă. Astfel ridicarea obiectului se realizează printr-un "RayCast" care pornește de la poziția pompei și se îndreaptă în direcția de jos a acesteia. Acest "RayCast" se duce până la o distanță maximă setată, detectând doar obiectele care fac parte dintr-un anumit strat setat, ridicarea realizându-se în momentul în care "RayCast" intersectează un obiect. Pentru "MoveObject()" este verificat dacă un obiect este ținut, și dacă este, funcția este apelată.

7.8 Sistemul de antrenare pentru operatorul uman

Pentru ca putea utilizatorul să poată să învețe să controleze brațul robotic s-a implementat o secțiune de tutorial. Acesta cuprinde o listă cu controalele brațului și cât o listă cu sarcini pe care să le execute utilizatorul pentru a învăța să folosească brațul.

7.8.1 Lista cu controale

Pentru lista cu controalele brațului a fost creat un JSON care cuprinde o structură de clase și vectori care cuprinde tipul butonului de pe manetă, care este butonul respectiv și ce acțiune realizează acest buton. Pentru citirea JSON-ului s-a creat un script nou ce conține 3 clase: clasa "JSONReader" care cuprinde partea de citire din JSON și afișarea conținutului acestuia în scenă, clasa "Controller" care cuprinde vectori de șiruri de caractere, ce conțin informații despre denumirea hardware a butonului și acțiunea asociată cu acesta din JSON și clasa "Controllers" ce cuprinde 2 vectori ce conțin elemente pentru maneta dreaptă, respectiv maneta stângă. Un aspect esențial pentru a putea citi din JSON este ca clasele "Controller" și "Controllers" să aibă atribuit atributul de "Serializable".

Pentru afișarea "JSON-ului" în scenă a fost folosit utilitarul TPro, și o componentă de tip TMP. Citirea "JSON-ului" se face în funcția "Start()" prin intermediul funcției "File.ReadAllText()" ce preia ca parametru un șir de caractere ce conține calea din sistem a fișierului JSON și returnează un șir de caractere cu conținutul fișierului.

Componenta "JSONUtility" din Unity preia conținutul obținut din funcția "File.ReadAllText()" îl deserealizează și realizează maparea conținutului pe structura celor 2 clase definite anterior. Toate datele citite sunt atribuite unei noi variabile de tip șir de caracter care la final este atribuită componentei TMP pentru a putea fi afișate în scenă.

7.8.2 Lista cu instrucțiuni de bază

Pentru partea de instrucțiuni de bază legată de funcționarea și controlul robotului s-a procedat în același mod: a fost creat un JSON cu acțiuni care trebuie executate, s-a creat un nou script care cuprinde clasa "Tutorial" care cuprinde vectori de șiruri de caractere, ce conțin informații despre acțiunea care trebuie executată, clasa "Tutorials" care cuprinde un vector de șiruri de caractere ce include elementele pentru tutorialul de bază, și "JSONScript" care cuprinde partea de citire din JSON și afișarea conținutului acestuia în scenă.

Însă pentru fiecare acțiune realizată s-a adăugat un indicator pentru a marca dacă acțiunea cerută a fost realizată sau nu. Acest indicator se bazează pe butoanele de scenă din Unity care își schimbă culoarea astfel: roșu dacă acțiunea nu a fost realizată, verde dacă acțiunea a fost realizată. Pentru acest indicator a fost creat un script nou ce are în componență următoarele funcții: "OnEnable()", "OnDisable()", "initColor()" care preia ca culoarea efectivă care este aplicată, "setColorProperties()" care preia parametru indicatorului din Unity, blocul de culoare, și cel de culoare efectivă "setColor()" care pe lângă parametrii de indicator din scenă, bloc de culori și culoare efectivă preia și ca parametru butonul de pe manetă care este apăsat, "setColorThumbstick()" care în loc de buton, preia ca parametru "thumbstick-ul" care realizează acțiunea, "setPumpColor()" care preia ca parametru un butonul , 2 butoane din scenă, blocul de culoare și culoarea efectivă, funcția "Awake()" care setează culoarea de bază a butoanelor și "Update()" în care sunt apelate funcțiile legate de culori.

Funcția "initColor()" în care sunt inițializate componentele necesare manipulării indicatorilor, acestea fiind: indicatorul folosit reprezentând o componentă de buton de Unity, blocul de culori pentru setarea culorilor pe componenta de buton, tipul de culoare care să fie aplicat, și aplicarea culorii pe indicatorul sau butonul din scenă respectiv.

Funcția de "setColorProperties()" este folosită pentru a seta proprietăți de culoare pe indicatoarele din Unity, acestea fiind:

- culoarea pe care o preia indicatoarele din Unity după cele de pe manete au fost apăsat, în acest caz fiind culoarea verde
- culoarea de evidență în momentul în care cursorul de la mouse se află peste acesta sau când maneta de VR este îndreptată către acesta fiind setată pe culoarea alb
- culoarea de tranziție a butonului între stările de neapăsat și apăsat fiind setată pe alb
- culoarea care marchează că butonul nu poate fi folosit fiind setată pe alb
- factorul de multiplicare al culorii care indică intensitatea culorii de pe indicatoare, este setată la valoarea 5 reprezentând o intensitate ridicată a culorii

La final deoarece modificările s-au realizat pe o copie a structurii culorilor se preia copia și se aplică pe indicatoarele din scena din Unity.

Funcția "setColor()" este folosită pentru a seta culoarea pentru acele indicatoare din Unity care reprezintă o acțiune executată prin intermediul butoanelor și "triggers" de pe manete. Aceasta verifică dacă butonul a fost apăsat, dacă a fost apăsat atunci este apelată funcția "setColorProperties()" cu parametrii necesari funcției.

Funcția `setColorThumbstick()` este folosită pentru a seta culoarea pentru acele indicatoare din Unity care reprezintă o acțiune executată prin `Thumbsticks` ale manetelor. Butoanele de `Thumbsticks` au fost separate deoarece acestea sunt văzute în Unity ca 2 variabile diferite. Principiul de funcționare este același ca cel al funcției `setColor()`.

Funcția `setPumpColor()` este folosită a reprezenta dacă acțiunea de activare sau dezactivare a pompei a fost realizată. Pentru reprezentarea acțiunilor de activare sau dezactivare în scenă au fost folosite 2 butoane. Prima dată se verifică dacă butonul a fost apăsat sau nu în cadrul curent față de celelalte funcții, unde se detecta o apăsare simplă. A fost folosit acest tip de verificare deoarece în Unity unui buton apăsat normal îi este detectată în același cadru din scenă mai multe apăsări, față de detecția în același cadru unde apăsarea este realizată o singură dată. Încă o necesitate a acestui tip de verificare este acela că pompa este activă la prima apăsare de buton și dezactivată la cea de a 2 a apăsare de buton.

Pentru a putea schimba culoarea pe indicatoarelor a fost folosit un contor care se incrementează în momentul când butonul de pe manetă a fost apăsat. La prima apăsare contorul devine 1 ceea ce înseamnă că pompa a fost activată și indicatorul specific își schimbă culoarea. La a 2 a apăsare se incrementează din nou contorul ceea ce înseamnă că butonul a fost din nou apăsat ceea ce înseamnă că pompa a fost dezactivată și indicatorul specific își schimbă culoarea. La final dacă au fost detectate mai mult de 2 apăsări se resetează contorul și indicatoarele își păstrează culoarea.

Funcția `Awake()` este folosită pentru a seta o culoare inițială pentru butoane. Aici este apelată funcția de `initColor()` care setează culoarea inițială pentru indicatoarele din scenă.

Funcția `Update()` este cea în care sunt setate culorile de realizare a activității. În aceasta sunt citite axele de pe `Thumbsticks`, este verificată dacă pentru acestea a fost trecut un anumit prag, prag setat datorită sensibilității acestora, după verificare este apelată funcția de `setColorThumbstick()`. Pentru restul butoanelor fiind apelată funcția `setColor()`, respectiv `setPumpColor()`.

7.9 Vizualizarea robotului din realitate în mediul virtual

Este o funcționalitate pentru a putea vizualiza robotul real în mediul din VR, pentru a valida că robotul real se mișcă în concordanță cu cel din mediul virtual și pentru al coordona spre obiectul care trebuie să-l manipuleze.

Scriptul ce se ocupă de această implementare folosește doar funcția `Start()` și o variabilă de tip `WebCamTexture` și `Render`.

În funcție, `WebCamTexture` este folosită pentru a putea identifica camera și a o accesa. `Render` este necesară a putea aplica imaginea obținută de la `WebCamTexture` pe o componentă din mediu pentru a o putea vizualiza.

7.10 Procesul de comunicație de date dintre Unity și cel din realitate

În această secțiune va fi prezentată comunicația între brațul virtual și cel din realitate, va fi explicat procesul de comunicație și cum valorile primite de la robotul virtual vor fi aplicate asupra brațului real și cum brațul real trimite comenzile recepționate către brațul virtual.

Comunicația dintre cei 2 roboți este realizată prin protocolul MQTT folosind un broker ce rulează pe calculatorul gazdă provenit de la HiveMQ și `tunneling` folosind ngrok care rulează de asemenea pe calculatorul gazdă. Fluxul comunicației între cei 2 fiind:

- Se inițializează brokerul MQTT și se pornește un tunel TCP folosind utilitarul ngrok către portul local al brokerului
- ngrok generează un URL și portul extern, care servește ca punct de acces universal prin internet către broker-ul local permițând conectarea ESP și Unity din rețele diferite

- Se realizează o conexiune între Unity și broker
- Unity transformă valorile provenite de la robotul virtual în valori relative care să fie aplicate pe robotul real în cazul motoarelor sau alte valori în cazul comenzii de reset sau pornire/oprire pompă
- Fiecare dintre aceste valori sunt publicate pe propriul topic și aceste topic-uri sunt transmise la broker
- Brațul real se conectează la broker
- Se abonează la topic-uri de unde preia datele și le parsează
- Datele parsate sunt preluate și aplicate pe braț
- Brațul publică la rândul său datele pe care le-a aplicat pe topic-uri pe care le trimite la broker
- Unity se abonează la acele topic-uri și se pot vizualiza ce date au fost aplicate pe braț

7.10.1 Comunicația din Unity

În această secțiune va fi prezentată cum Unity trimite unghiurile de la robotul virtual către robotul real. În acest caz Unity funcționează ca "publisher" și brațul real ca "subscriber".

În Unity a fost creat un nou script prin care să fie un "publisher". Acesta se folosește de următoarele funcții: "OnEnable()", "OnDisable()", "Start()", "Update()", "ConnectToBroker()", "SendRelativeData()", "SendReset()", "PublishInt()", "PublishFloat()", "SendNozzleValues()". Cu excepția funcției "OnEnable()", celelalte folosesc directiva "async" și cu excepția funcțiilor "OnEnable()", "Start()", "Update()" restul folosesc clasa "Task".

Funcția "ConnectToBroker()" este folosită pentru crearea clientului pentru Unity și conectarea la broker. Aceasta se folosește de o variabilă de tip "factory" cu care se poate crea clientul și de o variabilă de tip "ClientBuilder" prin care este specificată o serie de acțiuni specifică clientului și anume:

- conexiunea TCP cu server-ul un sunt precizare IP-ul și portul broker-ului
- numele clientului

După crearea clientului este apelată funcția pentru a se putea realiza conexiunea efectivă ci opțiunile setate.

Funcția "PublishInt()" este folosită pentru a putea publica valori întregi acestea fiind valorile unghiurilor provenite de la brațul virtual care vor fi aplicare pe cel real. Sunt folosiți 2 parametri: cel legat de topic-ul unde să publice și cel pentru valoarea pe care să o trimită, inițializate cu 0. Mesajul este construit printr-o variabilă de tipul "MessageBuilder()" cu următoarele opțiuni:

- numele topic-ului unde să publice
- și datele pe care trebuie să le trimită fiind necesare ca acestea să fie convertite la șir de caractere

După construirea mesajului variabila este trimisă către broker.

Funcția "PublishFloat()" este folosită pentru a putea publica valori cu virgulă acestea fiind valorile citite de la butoane. Sunt folosiți 2 parametri: cel legat de topic-ul unde să publice și cel pentru valoarea pe care să o trimită. Procesul de construire al mesajului și de trimitere al mesajului este același ca cel al funcției "PublishInt()".

Funcția "SendNozzleValues()" este folosită pentru a putea valorile de control pentru a putea controla pompa de sucțiune. Pentru aceasta sunt folosite 2 valori de tip bool: pentru a verifica dacă

pompa este activă și pentru a verifica dacă butonul care controlează starea pompei a fost apăsat în cadrul anterior. Aceste sunt necesare pentru asigurarea că butonul ce acționează pompa a fost apăsat doar o dată pentru a o activa, respectiv pentru a o dezactiva.

Astfel funcția începe prin a citi valoarea cu virgulă provenită de la buton și verifică dacă apăsare a depășit un anumit prag pentru a preveni apăsările accidentale. După se verifică dacă butonul a fost apăsat recent și nu a fost apăsat în cadrul anterior atunci variabila pentru a activa pompa este setată pe 1 ceea ce înseamnă că pompa a fost pornită și variabila pentru a verifica dacă pompa a fost activată este setată pe 1. Această valoare pentru pompă este trimisă printr-un topic separat către broker. Pentru oprirea pompei procesul este același. La final dacă butonul nu este apăsat variabila pentru a verifica dacă butonul a fost apăsat în cadrul anterior este setată pe 0.

Funcția "SendReset()" este folosită pentru a trimite comanda de reset provenită de la butonul atribuit acestei acțiuni. În aceasta este citită valoarea butonului de reset și trimisă folosind "Publish-Float()" către broker într-un topic propriu.

Funcția "SendRelativeData()" este cea folosită pentru a adapta valorile unghiurilor de la robotul virtual la valori care să fie aplicate către motoarele robotului real. Pentru a implementa această funcționalitate este necesară o citire inițială a pentru axele X, Y, Z care se realizează în funcția "Start()" unde de asemenea este apelată și funcția "ConnectToBroker()". După citirile inițiale se mai realizează încă o citire continuă a valorilor din interiorul funcției.

Folosind valorile citite sunt calculate valori delta pentru cele 3 axe astfel:

$$\text{delta}_X = -(\text{current}_X - \text{init}_X) * \text{sensitivity} \quad (9)$$

$$\text{delta}_Y = (\text{current}_Y - \text{init}_Y) * \text{sensitivity} \quad (10)$$

$$\text{delta}_Z = -(\text{current}_Z - \text{init}_Z) * \text{sensitivity} \quad (11)$$

unde, *current* reprezintă valoarea citită în funcție, *init* reprezintă valoarea citită inițial în funcția "Start()" și "sensitivity" reprezintă o constantă prin care valorile citite sunt adaptate pentru motoare. Valorile pentru X și Z sunt negative datorită modului cum este programat brațul real și anume pentru aceste axe valorile din real sunt oglindite față de cele din virtual.

Pe valorile absolute ale variabilelor delta se realizează o comparație cu o valoare de prag pentru a nu inunda microcontrolerul cu valori citite la intervale de zeci milisecunde, transmisia acestora putând duce la încetinirea sau chiar la oprirea sistemului.

Astfel prin această abordare sunt transmise acele valori care nu depășesc pragul. După verificări valorile sunt trimise la broker prin topic-uri individuale și valorile curente citite devin cele inițiale.

În funcția "Update()" sunt apelate funcțiile de trimitere ale valorilor astfel:

- "SendRelativeData()" și "SendNozzleValues()" sunt trimise la un interval de timp pentru ca microcontrolerul să poată să proceseze datele trimise.
- "SendReset()" este apelată numai când butonul de reset este apăsat și la același interval de timp

Pentru partea de feedback provenită de la brațul real, în Unity a fost creat un script prin care să devină un "Subscriber". Script-ul folosește 2 funcții principale: "Start()", "Subscriber()".

Funcția "Subscriber()" creează un nou client implementarea fiind aceeași cu cea din scriptul de "Publisher".

Pe urmă se ocupă de partea de primire a mesajului folosind o expresie lambda. Este creat un buffer pentru datele brute primite și o variabilă în care datele din buffer sunt decodificate, formate în șiruri de caractere. Deoarece expresia lambda se execută sincron nu așteaptă după alte procese și returnează un task deja completat pentru a satisface cerințele evenimentului. Funcția se conectează la broker și se abonează la topic-uri pentru datele primite de la brațul real.

Funcția "Start()" doar apelează funcția "Subscribe()"

Această implementare este cea finală pentru partea de Unity a proiectului. Mai jos vor fi explicate procesul de comunicație din robotului real și cum datele din Unity sunt aplicate asupra robotului real.

7.10.2 Comunicația din robotul real

Procesul de comunicare pe partea robotului real constă în următoarele operații:

- conectarea la o rețea WI-FI și la broker
- abonarea la topic-urile legate de datele de control provenite de la brațul virtual
- procesarea datelor primite
- aplicarea datelor pe motoarele brațului
- publicarea de topic-uri legate de datele care au fost aplicate

Conectarea la o rețea WI-FI a fost realizată printr-un script de micropython ce implementează o singură funcție și anume cea de "Connect". Funcția preia ca parametrii ai SSID-ul și parola rețelei, configurează microcontrolerul să funcționeze în modul de stație, pornește cipul fizic al microcontrolerul, se conectează la rețea folosind funcția "connect()" din biblioteca "network" și parametrii specificați.

Abonarea și publicarea la topic-uri cât și conectarea la broker este posibilă printr-un nou script ce le gestionează. Scriptul implementează următoarele funcții: "init()", "identificationInfo()", "Callback()", "connectToBroker()", "sub()", "pub()".

Funcția "init()" este folosită pentru a declara și inițializa variabilele pentru crearea clientului MQTT cât și variabile ce vor conține datele provenite de la topic-urile din Unity.

Funcția "identificationInfo()" este folosită pentru crearea și configurarea clientului MQTT al microcontrolerului. Folosește ca parametrii ID-ul clientului, adresa IP și portul brokerului. Creează o conexiune cu brokerul și redirecționează la funcția "Callback()" unde datele primite vor fi procesate.

Funcția "connectToBroker()" este folosită pentru conectarea la broker folosind clientul creat în funcția "identificationInfo()".

Procesarea datelor este realizată prin funcția "Callback()" constând din decodificarea din topic-uri, parsarea la tipul de dată necesar și atribuirea finală acestora pe variabilele care vor fi folosite pentru motoare a mesajelor.

Începe prin a decodifica topic-urile ce conțin datele, mesajele asociate cu acestea și parsarea din tipul de date "string" la tipul "float". Pe urmă este verificat dacă topic-urile decodificate cu cele la care să se aboneze sunt identice. În cazul în care sunt identice mesajul topic-ului este salvat într-o variabilă. Întreaga operație de procesare este realizată într-un bloc "try-catch" și când apare o eroare de de parsarea sau altele acestea vor fi afișate.

Pentru abonarea la un topic este folosită funcția "sub()" care are ca parametru topic-ul la care să se aboneze, iar pentru publicare este folosită funcția "pub()", care are ca parametru topic-ul la care să publice cât și mesajul propriu-zis.

Aplicarea datelor pe motoarele brațului este realizată în script-ul "main" al brațului cât și partea pentru feedback către Unity. Pentru partea de aplicare trebuie mai întâi conectată microcontrolerul la o rețea WI-FI și la broker-ul MQTT. Pentru WI-FI este apelată funcția "connect" din scriptul pentru conexiune și pentru broker "identificationInfo()" unde sunt specificate datele de autentificare necesare și "connectToBroker()" pentru conexiunea propriu-zisă.

După stabilirea conexiunilor este necesară abonarea la topic-uri pentru a prelua datele necesare. Aici este apelată funcția "sub" pentru toate topic-urile, acestea fiind pentru următoarele: motoarele care acționează baza secundare rotative, segmentul vertical principal și segmentul lateral de urcare-coborâre, resetarea la poziția inițială și acționarea pompei.

Următorul pas a constat în apelarea următoarelor funcții în momentul când robotul este alimentat și programat:

- "arm.go_home()" pentru a aduce brațul la poziția inițială setată de către producător
- setarea variabilei pentru starea pompei pe 0 pentru a o opri
- resetarea unghiului vârfului cu care sunt manipulate obiectele
- "arm.ORIGIN" setarea originii în coordonate carteziane X, Y, Z, aceste coordonate reprezentând cele 3 motoarele pentru mișcările stânga-dreapta, sus-jos, față-spate.

Pentru aplicarea valorilor pe motoarele a fost necesară o buclă "While" care să ruleze încontinuu. În această buclă sunt preluate din scriptul unde se procesează datele de la topic-uri în variabile separate. Aplicarea variabilelor pe motoare este realizată prin funcția "arm.set_position()" unde este calculată poziția unde trebuie să ajungă brațul prin adunarea la poziția curentă valoarea provenită din variabile.

Comanda brațului este executată într-un timp de 30 ms, poziția calculată devine noua poziție curentă și re setează variabila pentru ca brațul să nu se mai miște dacă nu primește valori noi sau dacă brațul nu poate ajunge fizic acolo. La final este folosit un temporizator în care motorul nu se mai poate mișca permițându-i microcontrolerului să realizeze alte operații.

Resetul brațului este realizată prin verificarea valorii topic-ului dacă a depășit un anumit prag. În cazul depășirii este apelată funcția "arm.go_home()" și coordonatele carteziane preiau valorile din variabila "arm.ORIGIN"

Acționarea pompei este realizată prin verificarea dacă variabila din topic depășește un anumit prag și în cazul în care acel prag este depășit este apelată funcția "on()" pe variabila pentru vârf în caz contrar funcția "off()" pe aceeași variabilă.

Partea de feedback este realizată prin publicarea pe topic-uri a valorilor folosite în calculul poziției și cele folosite la reset și la acționare pompă. Pentru poziții, valorile au fost preluate prin citiri ale coordonatelor carteziane în bucla "while" și o citire inițială în afara acesteia. Pe urmă au fost verificate dacă valorile absolut e dintre diferența celor 2 citiri depășesc un prag. Dacă depășesc acel prag atunci este construit un mesaj cu aceste coordonate, publicate pe topic și valorile citite inițial devin cele curente. Această implementare fiind necesară ca să fie trimise numai valori noi.

Pentru reset în momentul când această acțiune este realizată este construit topic-ul care conține mesajul prin care confirmă resetul și valoarea variabilei care a declanșat reset-ul. Pentru a transmite statusul pentru pompă a fost folosită o variabilă pentru a marca statusul acesteia, deoarece pentru a nu inunda Unity cu citiri continue putând duce la procesări din ce în ce mai lente ale datelor sau chiar la crash-ul întregului ambelor sisteme.

După ce pompa a fost acționată este verificată dacă status este diferită de 1. Dacă este diferită de 1 atunci este publicată în topic și variabila ia valoarea 1, marcând că pompa a fost pornită. În caz contrar la dezactivarea pompei se verifică dacă status este diferită de 0 și dacă este atunci este publicată în topic și variabila este setată la valoarea 0.

8.1 Importanța rezultatelor experimentale

Rezultatele experimentale sunt importante și necesare deoarece oferă informații obiective și irefutabile despre funcționalitatea și performanțele sistemului. Prin acestea se pot verifica și valida soluțiile implementate, optimiza și îmbunătăți sistemul și se pot evalua riscurile. Rezultatele experimentale ajută în validitatea teoriei și prezintă un suport în implementări ulterioare.

8.2 Testarea funcționalităților

Pentru testarea sistemului au fost realizate următoarele teste pentru:

- constrângerile cinematice și a bazei secundare rotative
- controlul brațului virtual folosind primul mod de control
- controlul brațului virtual folosind al doilea mod de control
- reset și manipulare de obiecte
- pentru sistemul de tutorial
- fluxul de comunicației între brațe

8.2.1 Testarea constrângerilor cinematice și a bazei secundare rotative

Constrângerile cinematice au fost testate prin dezactivarea "XRI Interaction Toolkit" și activarea controlului folosind tastatura. Astfel au fost folosite următoarele butoane: "W" și "S" pentru selectarea segmentului și "A" și "D" pentru rotirea într-o direcție sau alta a segmentului selectat.

Constrângerile cinematice au ca segmente părinte următoarele: segmentul vertical principal și segmentul lateral de urcare-coborâre. Baza secundară rotativă fiind părinte pentru restul segmentelor nu a necesitat constrângeri cinematice

Pe parcursul testării majoritatea constrângerilor cinematice au fost respectate cu excepția a tijei ajutătoare orizontale 1.

În cazul acestuia când segmentul lateral de urcare-coborâre se deplasa la unghiuri prea mari, acesta începea să se desprindă care ducea și la deviații ale suportului de pompă care începea să se deplaseze către interiorul brațului cum se poate observa în imaginea de mai jos:

8.2.2 Testarea controlului brațului virtual folosind primul mod de control

Primul mod de control al brațului a fost testat prin apăsarea fiecărui buton ale manetei care corespunde cu componenta care trebuia să o controleze. Astfel au fost testate următoarele acțiuni:

- thumbstick-ul când este mișcat stânga-dreapta baza rotativă să se rotească stânga-dreapta
- când sunt apăstate cele 2 butoane marcate cu litere să se ducă segmentul orizontal principal și cu suportul pentru ventuză sus-jos
- când sunt apăstate cele "triggers" segmentul vertical principal să se deplaseze stânga-dreapta

În urma acestui test acest mod de control s-a comportat foarte satisfăcător, executând corect și fără întârzieri acțiunile pe care trebuia să le execute în momentul în care au fost primite de la manetă.

8.2.3 Testarea controlului brațului folosind cel de al doilea mod de control

Al doilea mod de control a fost testat prin apăsarea butoanelor de pe manetă care controlează o direcție a efectorului final. În urma direcției a efectorului final segmentele brațului să se deplaseze în aceeași direcție. Astfel au fost testate următoarele direcții ale efectorului: sus-jos, stânga-dreapta, față-spate.

În urma testelor inițiale a fost constatat că brațul nu calcula unghiurile corect față de poziția efectorului final. Soluția la această problemă a reprezentat-o folosirea valorii de 90 negativă în calcularea unghiului pentru segmentul lateral. A mai fost observat faptul că baza secundară se rotește numai când brațul era complet întins, acesta trebuind să se rotească indiferent de calcule.

Soluția pentru cea de a 2 problemă a constat în separarea mișcării de rotație față de algoritm folosind controlul implementat la metoda directă.

Testele finale realizate cu soluțiile implementate au fost foarte satisfăcătoare, controlul fiind fluid și respecta direcțiile provenite de la efectorul final.

Butoanele care corespund cu direcțiile sunt aceleași cu care controlează direcțiile în cazul primului mod de control.

8.2.4 Testarea funcționalității de reset și manipulare de obiecte

Funcționalitatea de reset a fost testată prin apăsarea butonului atribuit acestor acțiuni. Pentru manipularea obiectelor testarea a fost realizată prin direcționarea brațului către obiectul din scenă și apăsarea butonului care este asociat cu ridicarea, respectiv coborârea obiectului.

Testul reset s-a comportat în conformanță cu așteptărilor. În momentul când a fost trimisă comanda de reset robotul a revenit la poziția inițială și unghiurile au fost setate la valori implicite.

În urma testării manipulării obiectelor a fost constatat faptul că dacă obiectul este lăsat în învecinătatea punctului de unde a fost preluat acesta se teleporta în acel punct.

O altă problemă minoră observată o reprezintă că în momentul în care este dată comanda ca pompa să fie dezactivată obiectul nu era lăsat din capătul ventuzei ci din punctul de origine al "jointului" care prinde ventuza de suport.

■ De pus imagine cu testele astea asta

8.2.5 Testarea sistemului de tutorial

Sistemul de tutorial a fost testat prin punerea în aplicare a acțiunilor legate de mișcarea prin scenă și cele de control al brațului. În urma execuției unei acțiuni de exemplu să se miște brațul stânga sau dreapta indicatorul corespunzător acelei acțiuni să se facă din culoarea roșu, care marchează că acțiunea nu a fost executată, în culoarea verde care marchează că acțiunea a fost executată.

În urma testării, s-a comportat foarte satisfăcător astfel încât când o instrucțiune din listă a fost executată, brațul a fost mișcat conform acțiunii respective și indicatorul din dreptul acțiunii respective și-a schimbat culoarea conform implementării.

8.2.6 Testarea fluxului de comunicație între brațe

Pentru verificarea că brațul real copiază mișcările celui virtual se baza pe afișarea în terminale a valorilor trimise de Unity, respectiv cele procesate de braț. Pe lângă a mai fost folosită metoda de testare prin validare vizuală externă unde a fost folosită o cameră video care filma brațul real când era mișcat cel virtual și erau comparate cele 2. Iar pentru sistemul de feedback testarea a fost realizată folosind consola din Unity unde a fost comparat cu ce a trimis Unity și cu ce recepționat Unity de la brațul real.

În urma testării inițiale a fluxului au fost constatate următoarele:

- brațul real se folosea de valori relative pentru a putea fi controlat nu valori absolute cum au fost transmise inițial valorile. Acest fapt a dus la plieri ale brațului aproape de limite, care au dus la ruperea ventuzei de suport.
 - De pus imagine cu asta
- poziția care stă robotul când este pornit nu are valoarea unghiului segmentului vertical principal 0 așa cum a fost presupus inițial
 - De pus imagine cu asta
- în momentul când se dorește ca brațul real să se deplaseze numai față-spate nu este posibil fără să fie coborât puțin la început
- a mai fost observat faptul că robotul real față de cel virtual se mișca sacadat adică avea în mișcare întreruperi scurte și bruște

În urma testării finale unde au fost remediate problemele legate de valorile trimise și fluxul de comunicație se comporta satisfăcător.

Valorile trimise erau aplicate corect pe braț și pentru problema de mișcare în care deplasarea brațul real față-spate nu este posibil fără să fie coborât puțin la început a fost constată că această implementare a fost aleasă de către producătorul brațului. Pentru partea de latență de comunicație din punctul de vedere al rețelei nu s-au făcut teste deoarece a fost folosită o rețea locală, unde este de așteptat latența este mică.

Rezultatul testului pentru sistemul de feedback a fost foarte satisfăcătoare în care valorile primite de a brațul real erau aproximativ aceleași cu excepția celei pentru pentru segmentul vertical principal.

- De pus imagini cu asta

8.3 Interpretarea rezultatelor testelor

Pe baza rezultatelor în urma testărilor realizate au fost identificate probleme provenite fie de la implementarea proprie realizată fie de la producător cât și posibile îmbunătățiri cât și ajustări care pot fi aduse asupra sistemului pentru a asigura o performanță optimă și o funcționare mai simplă și eficientă a sistemului.

Astfel, performanța sistemului din punct de vedere al procesării comenzilor primite de la manete căștii VR pentru robotul VR, de la trimiterea prin broker a datelor și aplicării acestora până la brațul real cât și trimiterea de feedback către Unity este peste așteptări.

8.3.1 Metrice, metri/secunda, filmand pas cu pas si script separat unde dau pasi de miscare gen coborare/urcare. fata/spate

Bibliografie

- [1] "The history and evolution of digital twin technology," Medium, Accessed: May 22, 2026. [Online]. Available: <https://medium.com/@sahilbhutada28/the-history-and-evolution-of-digital-twin-technology-6a0d97ee454b>
- [2] "What is a digital twin?" Intel, Accessed: May 28, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/learn/what-is-a-digital-twin.html>
- [3] "What is a digital twin?" IBM, Accessed: May 25, 2026. [Online]. Available: <https://www.ibm.com/think/topics/digital-twin>
- [4] "What is inverse kinematics?" MathWorks, Accessed: May 25, 2026. [Online]. Available: <https://www.mathworks.com/discovery/inverse-kinematics.html>
- [5] A. Hamad and B. Jia, "How virtual reality technology has changed our lives: An overview of the current and potential applications and limitations," International Journal of Environmental Research and Public Health, vol. 19, no. 18, 2022, ISSN: 1660-4601. DOI: 10.3390/ijerph191811278 [Online]. Available: <https://www.mdpi.com/1660-4601/19/18/11278>
- [6] "What is virtual reality (vr) and how does it work?" TEAM VIEWER, Accessed: Feb. 19, 2026. [Online]. Available: <https://www.teamviewer.com/en/solutions/use-cases/virtual-reality-vr/>
- [7] "Mqtt: The standard for iot messaging," MQTT, Accessed: Apr. 25, 2026. [Online]. Available: <https://mqtt.org/>
- [8] "What is pub/sub messaging?" AWS, Accessed: Feb. 19, 2026. [Online]. Available: <https://aws.amazon.com/what-is/pub-sub-messaging/>
- [9] "Intro to ros," CLEARPATH ROBOTICS, Accessed: Feb. 19, 2026. [Online]. Available: <https://www.clearpathrobotics.com/assets/guides/kinetic/ros/Intro%20to%20the%20Robot%20operating%20System.html>
- [10] "What is urdf (unified robot description format)?" FOXGLOVE, Accessed: Feb. 19, 2026. [Online]. Available: <https://foxglove.dev/robotics/urdf>
- [11] "What are industrial robots?" MABUCHI-MOTOR, Accessed: Feb. 19, 2026. [Online]. Available: <https://solution.mabuchi-motor.com/en/blog/8>
- [12] "6 types of industrial robots," Robotec, Accessed: Feb. 19, 2026. [Online]. Available: <https://robotec.org/blog/types-of-industrial-robot.html>
- [13] What is tunneling? | tunneling in networking, CloudFlare, 2026. Accessed: Jun. 3, 2026. [Online]. Available: <https://www.cloudflare.com/learning/network-layer/what-is-tunneling/>
- [14] "What can unity do?" Unity, Accessed: Feb. 18, 2026. [Online]. Available: <https://learn.unity.com/course/unity-for-artists-curricular-framework-resources/tutorial/what-can-unity-do#5fa1bdc0edbc2a01f0fae7a5>
- [15] "Xr interaction toolkit," Unity, Accessed: Apr. 25, 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/index.html>

- [16] "Articulation body component reference," Unity, Accessed: Apr. 25, 2026. [Online]. Available: <https://docs.unity3d.com/6000.3/Documentation/Manual/class-ArticulationBody.html>
- [17] "Articulationbody," Unity, Accessed: Apr. 25, 2026. [Online]. Available: <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/ArticulationBody.html>
- [18] K. Paul, "Raycast in unity," 2024. DOI: 10.1109/ICCAR.2015.7166008 Accessed: May 28, 2026. [Online]. Available: <https://medium.com/@pkillman2000/raycast-in-unity-4c1895dc33d6>
- [19] "Rigidbody," Unity, Accessed: May 28, 2026. [Online]. Available: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Rigidbody.html>
- [20] "Urdf-importer," Unity-Technologies, Accessed: Apr. 25, 2026. [Online]. Available: <https://github.com/Unity-Technologies/URDF-Importer>
- [21] "What is autodesk fusion?" Autodesk, Accessed: Apr. 25, 2026. [Online]. Available: <https://www.autodesk.com/solutions/what-is-fusion-360>
- [22] What is ngrok? SendBird, 2026. Accessed: Jun. 3, 2026. [Online]. Available: <https://sendbird.com/developer/tutorials/what-is-ngrok>
- [23] "Hiwonder maxarm open source robot arm powered by esp32 support python and arduino programming inverse kinematics learning," Hiwonder, Accessed: Apr. 25, 2026. [Online]. Available: <https://www.hiwonder.com/products/maxarm?variant=40008714092631>
- [24] "Hardware basic learnig," Hiwonder, Accessed: May 31, 2026. [Online]. Available: https://docs.hiwonder.com/projects/MaxArm/en/latest/docs/5.Hardware_Basic_Learnig_formatted.html
- [25] "Meta quest 3s," Meta, Accessed: Apr. 25, 2026. [Online]. Available: <https://www.meta.com/quest/quest-3s/>